

Secdrop-Secure Encrypted File Sharing Platform

Team Members

Dilpreet Sidhu - dss580@uregina.ca

Gurpreet Kaur- gkr158@uregina.ca

Priyanshu- ptc260@uregina.ca

Jenil Chaklasiya-jco149@uregina.ca

1. Problem Definition

Secdrop is a secure way for individuals, startups, and nonprofits to send encrypted files over the internet. It is easy to set up and use and puts a lot of emphasis on user privacy. There are two categories of users: Admin Users and General Users. Every role comes with its own collection of tools for handling platform settings, sharing, and file access. The platform employs AES-256 encryption, allows for password security, utilizes token-based sharing, and supports anonymous uploads to ensure that key assets remain protected from unauthorized access throughout their entire lifespan.

In the digital age, file sharing needs to be safe and private. It is risky to use cloud storage services like Google Drive and Dropbox or email to share files as follows:

- It does not offer default end-to-end encryption. Thus, other people can access the data.
- It has complicated user interfaces which imposes difficulty for non-tech-savvy customers.
- It offers paid plans for more security.

Because of these limitations, we need a free, easy-to-use, safe, and privacy-focused platform that doesn't lose usability.

Secdrop offers a secure and free method for sharing files without the need for payment for full access or integrated encryption. It tackles the increasing need for privacy in file sharing. Secdrop focuses on both security and usability, offering AES-256-GCM encryption, password protection, anonymous uploads, and role-based access. The web-based interface makes setup super easy, and features like token-based sharing and QR code generation give users a lot of control. Secdrop allows people to share sensitive files securely without depending on expensive or centralized options.

2. Application Benefits

File sharing solutions allow for collaboration, remote access, and information exchange in today's digital world. Even though Dropbox and Google Drive are packed with features and are super popular, they really struggle with privacy, encryption, and ease of use.

Secdrop is compared to Dropbox and Google Drive as a secure, lightweight file-sharing service that offers encrypted, anonymous, simple data exchange.

Dropbox:

Link- [Login or Sign Up - Dropbox](#)

Dropbox lets you store files securely and share them through links, making collaboration easier and faster. It works well with productivity apps like Microsoft Office, making it easier to manage documents and improve workflow. Dropbox lets users access the latest versions of their files from any device by automatically syncing everything across multiple devices.

There are numerous issues with Dropbox, despite its immense popularity. Users must establish an account to submit and retrieve files, which can be challenging when attempting to maintain their privacy while sharing data. End-to-end encryption isn't turned on by default, which might put data security at risk. Also, access control is a feature that's only found in the pricier plans.

Secdrop fills the gap and limitations of Dropbox as follows:

1. Fully **anonymous file sharing is supported**; no account creation or registration is necessary.
2. **By default, AES-256-GCM encryption is activated** during the upload and download processes.
3. **Admin-level controls** are available for **free**, in contrast to Dropbox, where they are restricted to premium plans.
4. Access to **password-protected and token-protected files** without **disclosing their identity**.

Google Drive:

Link-[Home - Google Drive](#)

Google Drive makes it easy for people to get their files on Windows, macOS, Android, and iOS devices. Real-time collaboration is quite easy when you connect to products like Google Docs, Sheets, and Slides. This lets many people work on the same document at the same time. It also allows users to manage their access levels by enabling them to configure specific permission settings for file sharing based on links.

There are certainly some substantial privacy and security concerns associated with Google Drive. Users must login with a Google account to access any features, which complicates the experience for those who prefer to remain anonymous. Additionally, the data may be in danger to exposure due to the fact that the files that are stored and shared on the platform are not automatically encrypted from end to end. Individuals' identities are revealed when they share files, which complicates the process of sharing files anonymously.

Secdrop offer key benefits compared to Google Drive as follows:

1. There is **no need to create an account** or share your name to access it.
2. **AES-256 end-to-end encryption** really makes sure that data is protected and keeps your privacy intact.
3. **Sending files** while keeping your **identity hidden** through password or token-based connections.

Secdrop bridges a gap for individuals and organizations with high security requirements but limited resources by offering a secure file sharing platform which is accomplished by safeguarding the privacy of its users. The following are a few of the unique features of Secdrop:

1. End-to-end encryption ensures that files are not visible or edited by anyone else when they are stored and shared.
2. It makes it easier to share files without a need to download or register for software.
3. Sharing is protected by a password and code, and users are only permitted to access it if they have valid credentials.
4. Anonymous uploads are permitted.
5. The interface is super user-friendly, resulting in an increase in accessibility.

In conclusion, Secdrop tackles the gaps identified in file sharing services such as Dropbox and Google Drive as it is user-friendly and promotes security during the file sharing process. Dropbox and Google Drive might be the popular options that offer cloud storage and collaboration features. However, they aren't designed for secure and private file sharing.

3. Requirement's Elicitation and Specification

3.1 Functional Requirements (General User)

- **Upload File(s)**
 - Select or drag-and-drop one or more files (no registration required).
 - Client-side AES-256-GCM encryption with a 7-digit passphrase.
 - Enforces file-size and type limits (configured by Admin in future).
- **Generate Secure Share Link**
 - On successful upload, user receives:
 - A unique download URL (/download/<fileId>)
 - A QR code for that URL
 - Links contain no user identity or raw data.
- **Request One-Time Code (OTP)**
 - Request a six-digit OTP to be sent to the specified email or phone.
 - OTP stored temporarily, delivered via email/SMS.
- **Download & Decrypt File**
 - Enter the 7-digit passphrase and OTP on the download page.
 - On valid credentials, server streams and decrypts the file via AES-256-GCM and initiates download.
- **Receive Error Feedback**
 - Clear, specific messages for missing/invalid inputs (e.g. no file, bad passphrase length, malformed email/phone, wrong OTP, expired link).

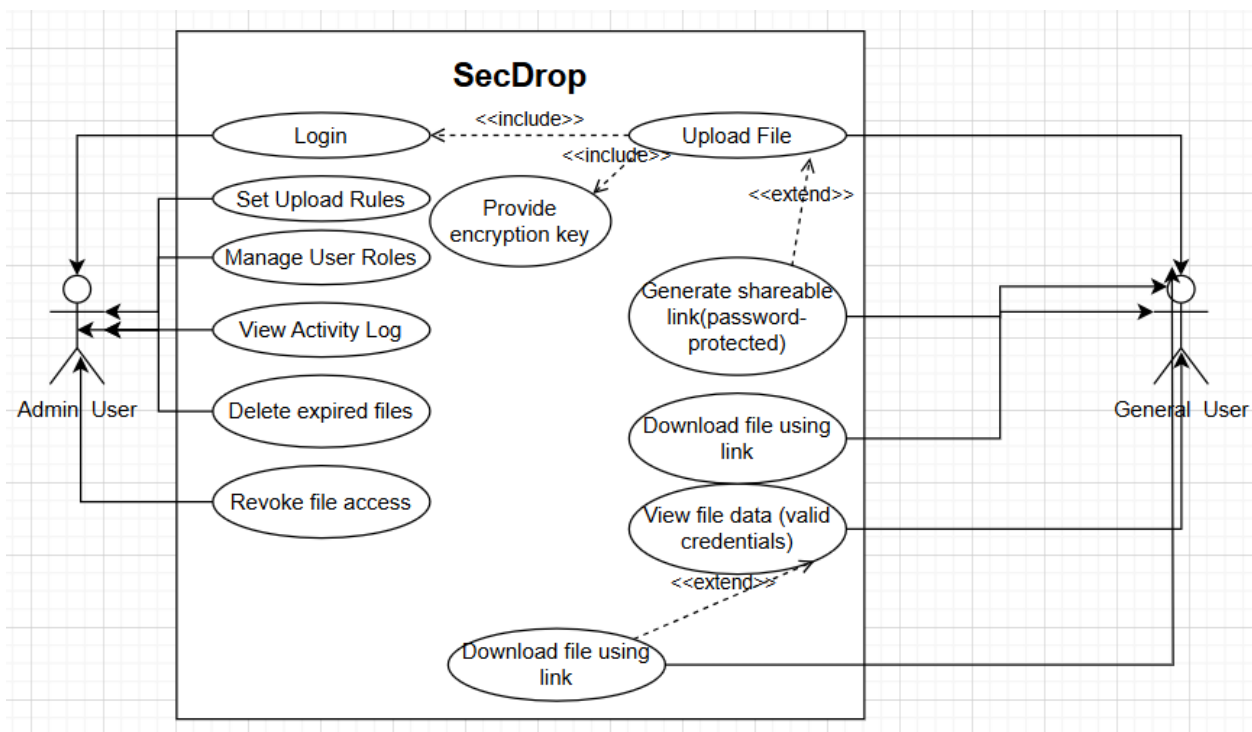
- Blocks further action until corrected.

3.2 Future Work: Admin Role

The following Admin capabilities are planned for a future release:

- **Configure Upload Policies**
 - Set maximum file size, allowed file types, and default expiration periods.
- **Manual File Deletion**
 - Remove encrypted files and metadata by file ID to free storage or revoke access.
- **Activity Log Monitoring**
 - View and filter timestamped records of uploads, OTP requests, and downloads for auditing purposes.

3.3 Use Case Diagram



3.4 Activity Diagram for Complex Use Cases

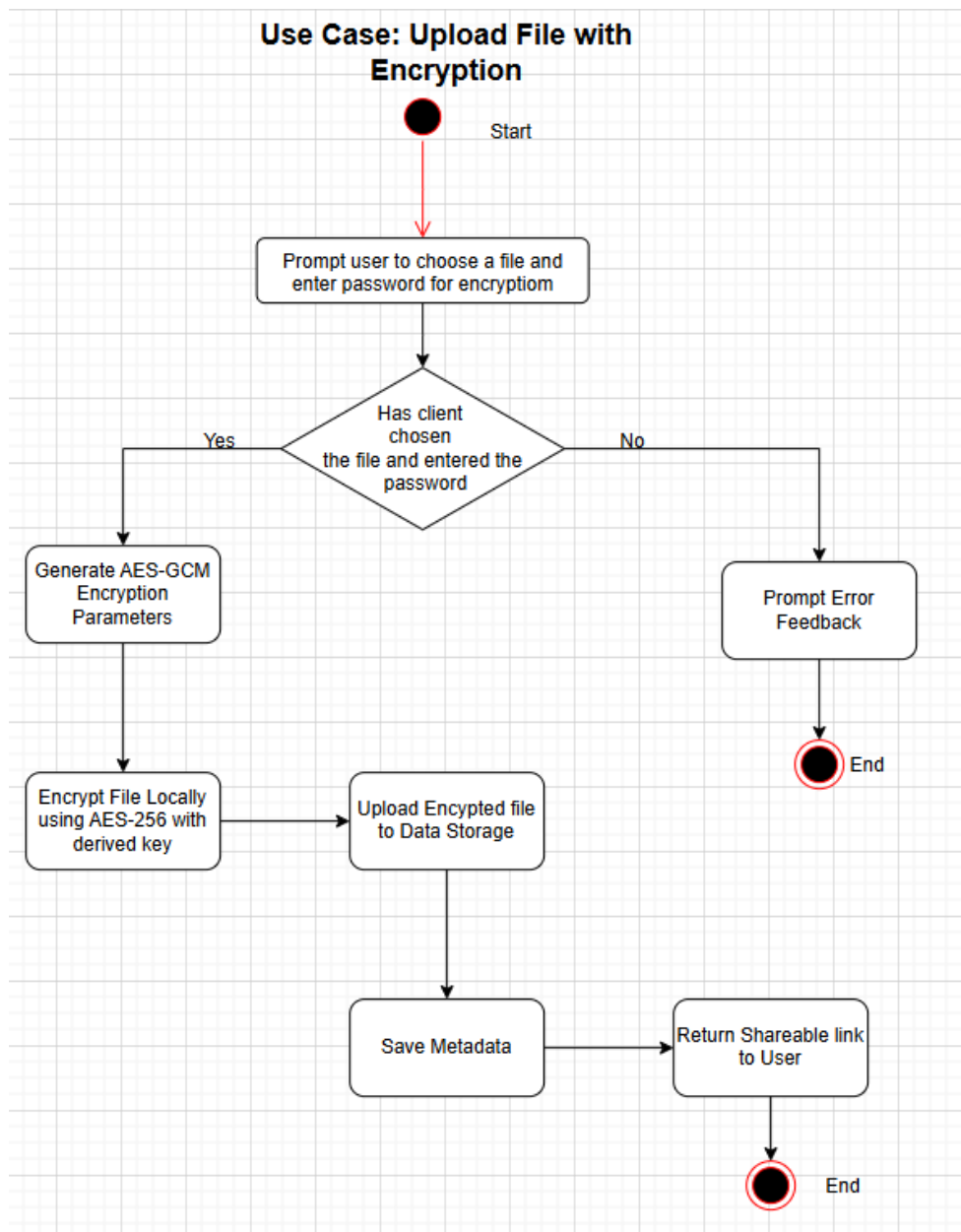


Figure 1: Use Case-Upload File with Encryption (General User)

USE CASE 1: Upload File

To safeguard sensitive information, the objective is to make certain that the files are uploaded in a secure manner and that they are stored in an encrypted format. On the web platform, the user accesses the interface necessary for uploading files. In addition to selecting the file, the user may also choose to set up the password or token. When the file is sent, it is encrypted automatically by default. After that, the file that has been encrypted is uploaded to the S3 bucket. To deliver it to the recipients, the link that can be shared is generated.

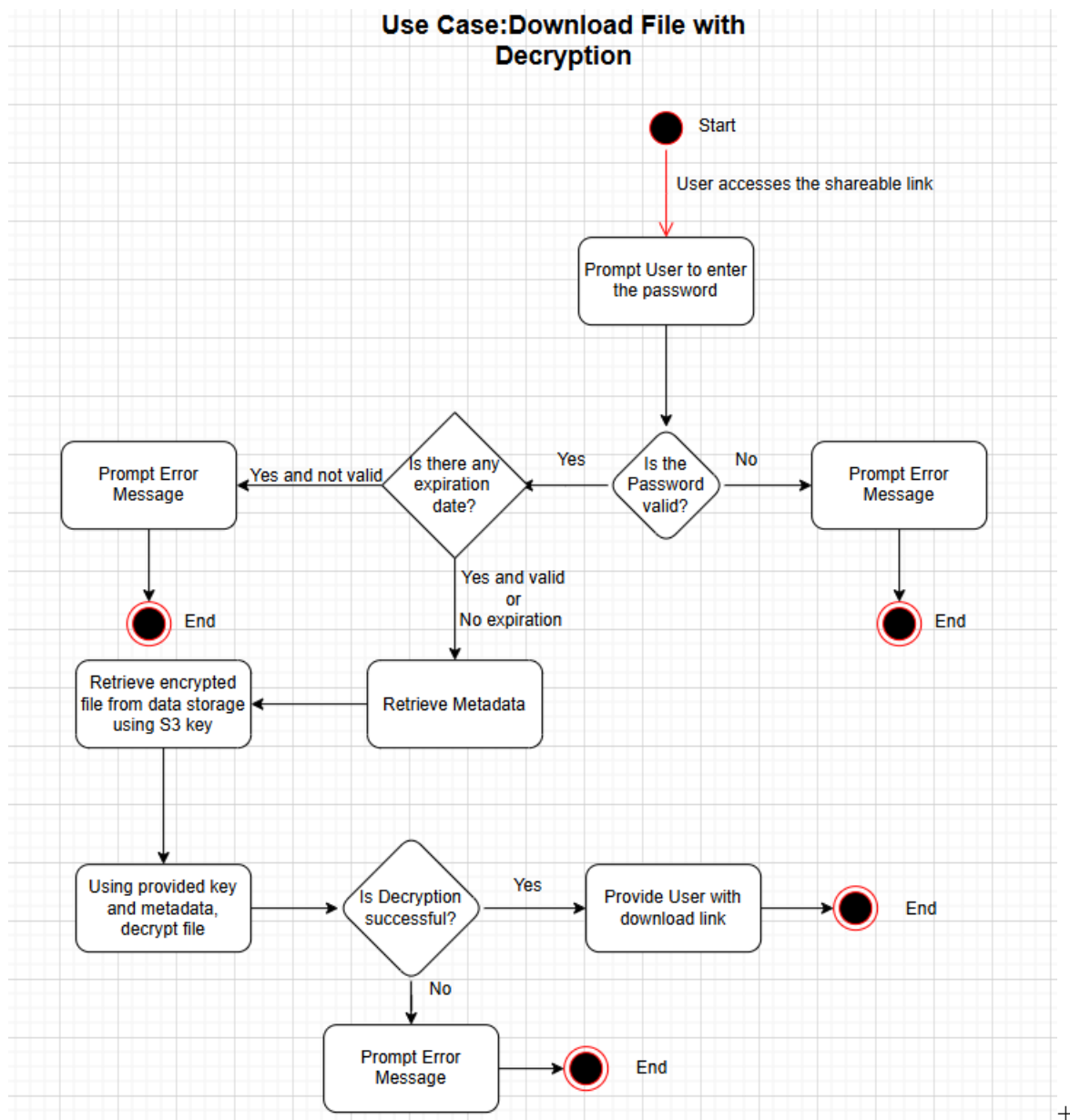
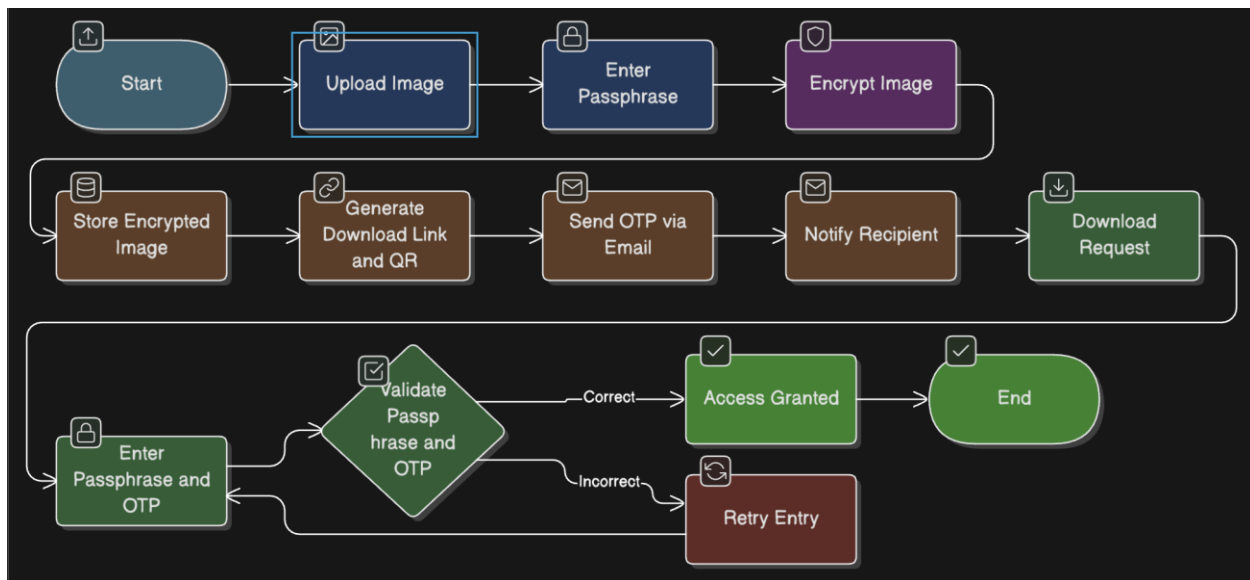


Figure 2: File Decryption and Download (General User)

Use Case 2: Download File with Decryption

The objective is to make it possible for authorized users to retrieve encrypted files, ensuring that the confidential information is safeguarded during the downloading process. Through the usage of the shared link, the user is able to access the file download interface that is located on the web platform. If it is necessary, the system will request that the user provide the token or credentials. Decryption of the file is performed with AES-256-GCM, and the user then downloads the file.

Summary of process:



3.5 Software Qualities for Secdrop

- **Correctness** This Software Quality means that the system is working as intended in its functional requirements and produces the expected results. For Secdrop, it ensures that the platform delivers reliable outcomes for admin and general users according to their roles and permissions.
 - Example 1: When a general user uploads the file and applies the password, the system ensures that the error feedback prompts to enter the correct password. Any incorrect password attempt denies access, and appropriate error feedback is displayed. This guarantees that the file is secure and only accessible to intended users.
 - Example 2: If a user sets a expiration date while uploading the file, the system must track the date and automatically make it inaccessible once the date and time are reached. Any attempt to download the file after the expiration should result in error feedback indicating that the file has expired. This guarantees that file availability adheres to user-defined constraints.
- **Time Efficiency** This critical software quality ensures system responsiveness and prompt execution of key operations. For Secdrop, it is to ensure that the key operations are performed with minimal delay such as uploading, downloading, and encryption of the files as it enhances the user experience.

- Example 1: General users can upload large size files that are encrypted by default using AES-256-GCM with minimal delay. The encryption process is optimized to work in real time so that the user doesn't have to wait a long time, enhancing the user experience.
- Example 2: When a user initiates the file download by clicking on shareable link, the system retrieves the file and decrypts the file using the password if needed. The decrypted file is then made available to the user to download to the user's device. This increases software optimization and low latency architecture.
- **Robustness** This is an external software quality which defines the expectations of the software when unexpected situations and incorrect inputs are entered by the user. The program should have the ability to tackle these situations without crashing and giving incorrect output. In reference to Secdrop, it plays a vital role in maintaining data security and providing great experience for both admin and general users.
 - Example 1: When a general user attempts to upload a file with a type that is not supported, the system validates the file type before the upload process. If the file type is not supported, the user receives the error feedback explaining the issues and requesting to correct it. This helps to prevent potential security issues.
 - Example 2: If a user is trying to access the download link after expiration date or uses the invalid link, the system prompts the error feedback stating that the file is no longer available and rejects the request. This ensures that the expired files remain unavailable as intended by the uploader and adhere to the policies.

4. Architecture & Design Patterns

4.1 Three-Layer Architecture

SecDrop follows the Model–View–Controller (MVC) pattern to separate data management, user interface, and request handling:

Model

- Encryption Service (server/filesencryptdecrypt.js)
 - Derives AES keys (PBKDF2), handles AES-GCM encrypt/decrypt pipelines, and manages in-memory metadata (IVs, auth tags).
- Storage Service (server/ec2Storage.js)
 - Abstracts S3 uploads/downloads, so the rest of the code doesn't care whether files live on disk or in the cloud.
- ID & Token Helpers (server/base58.js, crypto.randomUUID())
 - Generate human-friendly file IDs and one-time passcodes without exposing implementation details.

View (User Interface)

- React Components (client/src/*.js, .css)
 - FileEncryptUpload.js – Multi-file drag-and-drop, passphrase input, progress bar, and QR-code generation.
- DownloadPage.js – Passphrase & OTP form and file download link.
 - Navbar.js, Homepage.js, About.js, DonationPage.js – Navigation and static pages.
- Setup (client/src/setupProxy.js)
 - Proxies /api calls to the Express backend during development.

Controller (HTTP Routing & Workflow)

- Express App (server/server.js)
 - Initializes middleware (CORS, JSON parser, Multer), mounts routes, and starts the server.
- Route Definitions (server/routes/files.js)
 - POST /api/crypto/upload → uploadFile() (Model)
 - GET /api/crypto/metadata/:id → getMetadata()
 - POST /api/crypto/request-otp/:id → requestOtp()
 - POST /api/crypto/decrypt/:id → downloadFile()

Models (Data & Business Logic)

- server/filesencryptdecrypt.js
- server/ec2Storage.js
- server/base58.js
- server/routes/files.js (*wraps model handlers*)

Views (UI & Presentation)

- client/src/App.js
- client/src/App.css
- client/src/index.js
- client/src/index.css
- client/src/logo.webp
- client/src/Navbar.js
- client/src/about.js
- client/src/DonationPage.js
- client/src/Homepage.js

- **client/src/Homepage.css**
- **client/src/FileEncryptUpload.js**
- **client/src/FileEncryptUpload.css**
- **client/src/DownloadPage.js**
- **client/src/DownloadPage.css**

Controllers (HTTP Routing & Workflow)

- **server/server.js** (*main Express app*)
- **server/routes/files.js** (*upload/download/OTP routes*)
- **server/p2p/signaling.js** (*optional WebRTC signalling*)
- **client/src/setupProxy.js** (*dev-time proxy for API calls*)

Three-layer architecture promotes the separation of concerns as each layer has its own responsibility. As a result, it simplifies the process of development and maintenance. It makes the process of debugging easier as well since the error can be traced and corrected without impacting the other layers.

Also, the modularity of the layers allows the components to be reused throughout the system. For example, UI components such as upload or download buttons can be reused throughout the system for different purposes. This reduces redundancy in code and promotes consistency.

Since sensitive key operations such as encryption, decryption and password verification are isolated in the logic layer, which prevents direct user access. This also increases security by minimizing the risk of exposing critical code and information to attackers. This promotes the core concept of security and robustness.

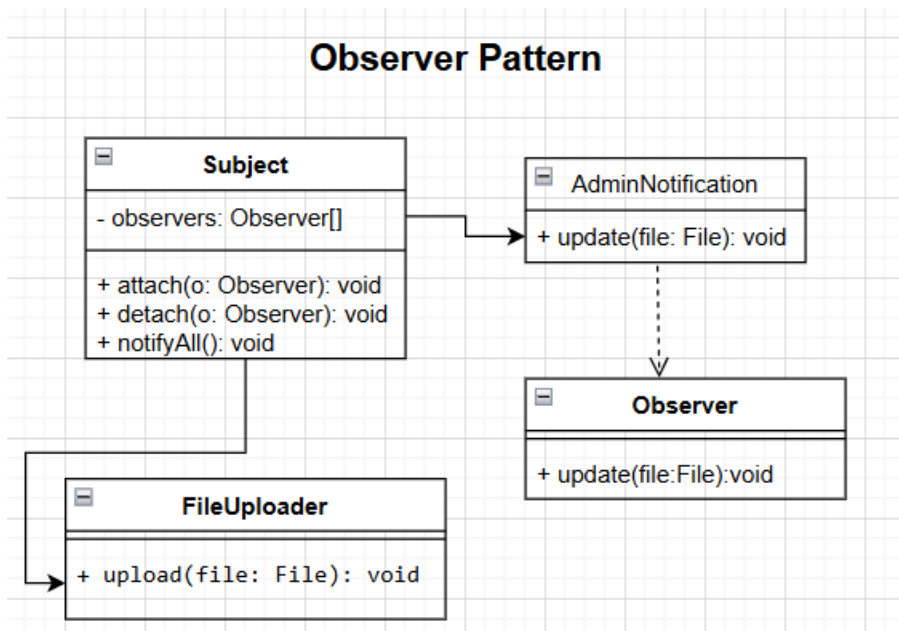
To summarize, the adoption of three-layer architecture in Secdrop made the process of development and maintenance easier. It also enhances the system's performance, scalability, and security. It also works best for Secdrop as the core concept is secure file sharing which is promoted by clear separation and modularity.

4.2 Design Patterns

OBSERVER PATTERN

This is a behavioral design pattern that allows for one-to-many dependencies between objects. The changes made on one subject update all other observers. This design pattern enhances the flexibility and extensibility of software. In Secdrop, the observer pattern handles real-time activities such as monitoring activities for the admin log. The observer can capture and reflect the changes without modifying core logic. For example: when the general user uploads the file, it

notifies the admins(observers) about the changes but doesn't modify the upload logic. Below is the class diagram for the design pattern:



Location: /server/p2p/signaling.js

The pattern logic is used in the software to support multiple admins (Observers) dynamically promoting extensibility. The usage of the observer's pattern aligns with the three-layer architecture as it also promotes loose coupling, scalability, and maintainability, which makes it ideal for security-sensitive operations to be tracked without interfering with core logic.

FACTORY PATTERN

The factory pattern is a creational design pattern that provides a way to encapsulate objects. Factory pattern uses a separate method to create instances of objects without exposing the logic of object creation. Use in Secdrop: The password/token generator in `server/filesencryptdecrypt.js` (or a dedicated helper) produces strong, policy-compliant tokens for shareable links. Client code simply calls `TokenFactory.create()` without handling length, character sets, or uniqueness .

- Generate strong password/tokens
- Ensures that password adheres to system policies
- Password logic is hidden

```

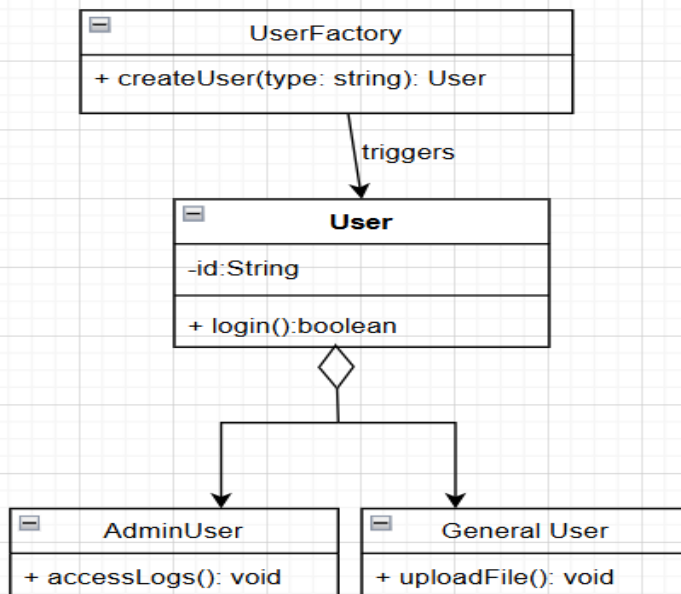
// Generate a one-time password
function generateOTP() {
  return Math.floor(100000 + Math.random() * 900000).toString();
}

// Generate a unique file identifier
const id = crypto.randomUUID();

// Use the factory to get a fresh OTP
const otp = generateOTP();

```

Factory Pattern



The benefit of using the factory pattern is the confidentiality of logic and maintain security by centralizing the password creation, which is the core concept of the software. It also strengthens the modularity and enhances the consistency and reusability of the software.

SINGLETON PATTERN

This is a creational design pattern that makes sure a class has only one instance to maintain a consistent operational environment. A Config class (or a module exporting a single object) centralizes settings like AES parameters, file-size limits, and expiration defaults. All modules import this singleton, guaranteeing consistency across encryption, upload constraints, and admin panels.

- Encryption standards
- Upload constraints such as size limits, expiration settings
- Administrative controls such as enabling or disabling password-protection

Singleton Pattern

UploadTracker
- static instance: UploadTracker - uploads: Map<string, Metadata>
+ static getInstance(): UploadTracker + addUpload(id: string, meta: Metadata): void + getUpload(id: string): Metadata

```

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const multer = require('multer');
const path = require('path');
const fs = require('fs');
const { uploadFile, downloadFile, getMetadata, requestOtp } = require('./filesencryptdecrypt');

// Ensure tmp directory
const tempDir = path.join(__dirname, 'tmp');
if (!fs.existsSync(tempDir)) fs.mkdirSync(tempDir, { recursive: true });

const app = express();
app.use(cors());
app.use(express.json());

const upload = multer({ dest: tempDir });
app.post('/api/crypto/upload', upload.array('files'), uploadFile);
app.get('/api/crypto/metadata/:id', getMetadata);
app.post('/api/crypto/request-otp/:id', requestOtp);
app.post('/api/crypto/decrypt/:id', downloadFile);
app.get('/', (req, res) => res.send('SecDrop backend is running.));

const port = process.env.PORT || 4000;
app.listen(port, () => console.log(`SecDrop backend listening on port ${port}`));

```

The singleton pattern helps to maintain consistency and eliminate discrepancies that can occur in the software. For example, having multiple instances of file size limits or encryption can lead to undefined errors. Also, state information such as admin defined policies remains persistent, verifying that the components are operating with the latest version. If one instance is pertained, it

also prevents the potential bugs stemming from the system settings. Since it avoids redundant object creation, it saves system resources which enhances the time efficiency of the system.

FACADE PATTERN

This is a structural design pattern that simplifies the interface of a complex subsystem. It also hides the complexities from external modules by acting as a wrapper and providing a clean and modular design as the system delegates these tasks to the modules through a facade. Use in Secdrop:EncryptionFacade in server/filesencryptdecrypt.js wraps the details of PBKDF2, AES-GCM IV/auth-tag handling, and stream pipelines behind two methods: encryptStream(input, password) and decryptStream(id, password). Express route handlers (server/routes/files.js) use this facade, keeping controllers thin and focusing solely on HTTP concerns .

```

async function downloadFile(req, res) {
  const { id } = req.params;
  const { keyB, otp } = req.body;
  const meta = uploads[id];
  if (!meta) return res.status(404).json({ error: 'Invalid file ID' });
  if (!otp || otp !== meta.otp) {
    return res.status(401).json({ error: 'Invalid verification code' });
  }

  try {
    const key = await deriveKey(keyB);
    const iv = Buffer.from(meta.ivHex, 'hex');
    const authTag = Buffer.from(meta.authTagHex, 'hex');
    const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
    decipher.setAuthTag(authTag);

    res.setHeader(
      'Content-Disposition',
      `attachment; filename="${encodeURIComponent(meta.originalName)}"`
    );
    res.setHeader('Content-Type', 'application/octet-stream');
    console.time('Download & Decrypt');
    await pipelineAsync(
      downloadFromS3(meta.s3Key),
      decipher,
      res
    );
  }
}

```

```

// Handler: Encrypt, upload to S3, and notify with link (no OTP)
async function uploadFile(req, res) {
  const files = req.files || [];
  const { keyB, recipient } = req.body;

  if (!files.length || !keyB || !recipient) {
    return res.status(400).json({ error: 'Missing files, passphrase, or recipient' });
  }

  // Generate a one-time password
  function generateOTP() {
    return Math.floor(100000 + Math.random() * 900000).toString();
  }

  // Generate a unique file identifier
  const id = crypto.randomUUID();

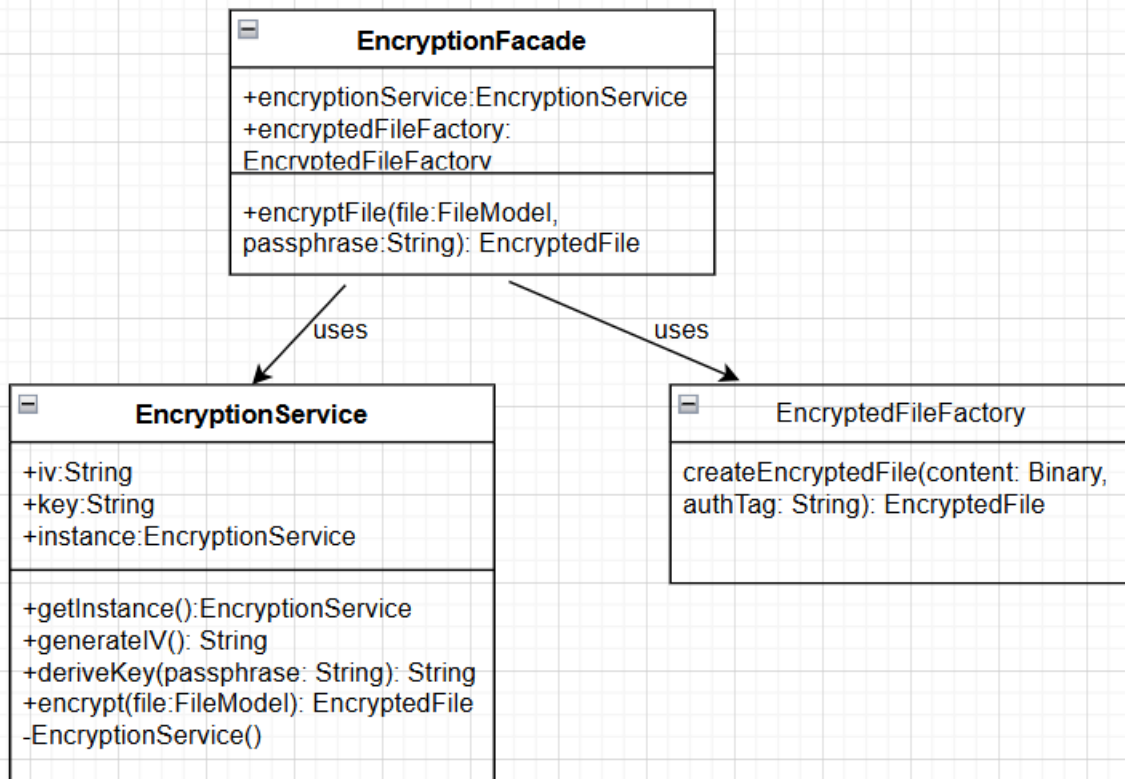
  // Use the factory to get a fresh OTP
  const otp = generateOTP();

  const results = [];

  for (const file of files) {
    const id = crypto.randomUUID();
    const iv = crypto.randomBytes(16);
    const key = await deriveKey(keyB);

```


Facade Pattern



Location:

-/server/filesencryptdecrypt.js

- /server/routes/files.js

```
//defining routes
router.post('/upload', upload.single('file'), uploadFile);
router.post('/decrypt/:id', downloadFile);
module.exports=router;
```

Lastly, the use of facade promotes modularity as its breakdown of complex logic into smaller, reusable components. Since the complex subsystem is broken down into smaller components, it

also improves maintainability and enhances the separation of concerns. The separation of concerns simplifies unit testing as encryption, decryption, or upload logic can be tested independently. Thus, it enhances developer experience and long-term code sustainability.

MVC Pattern

The Model-View-Controller (MVC) design pattern is an architectural design pattern which is widely employed to separate concerns within applications. It divides the application into three components:

1. **Model:** This layer manages the data, business logic, and rules of the application.
2. **View:** This layer represents the user interface that displays data to user and captures user interactions
3. **Controller:** This layer is a mediator between View and Model layers. It processes user inputs, manipulates data via Model, and updates the View accordingly.

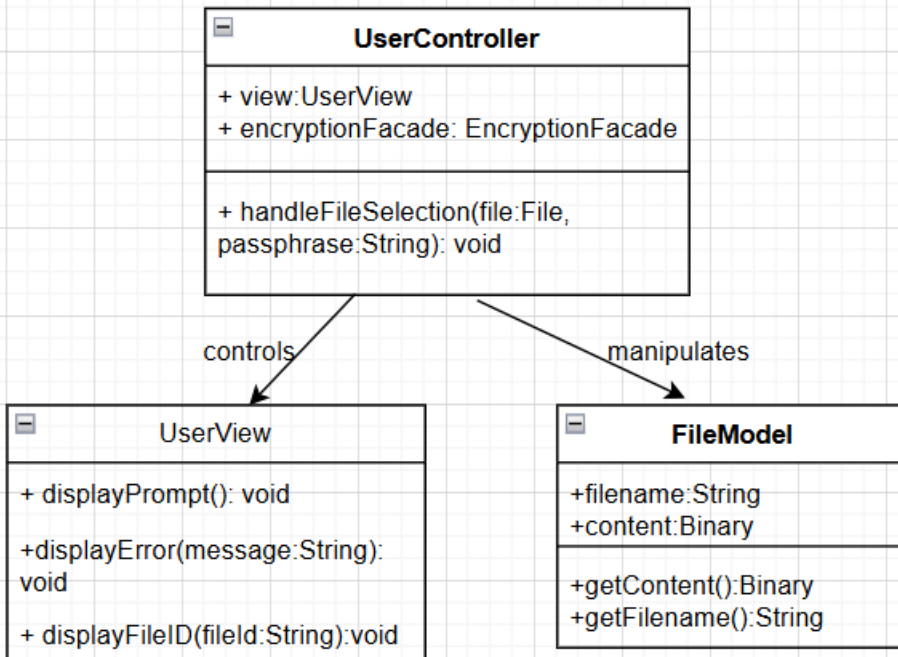
In Secdrop, the MVC pattern is employed to enhance modularity, scalability and maintainability as follows:

1. **Model:** This layer handles file metadata such as file IDs, passwords, encryption keys etc., user roles, and business roles such as file expiration and size limits.
2. **View:** This layer implements the UI interface using HTML, CSS, and JavaScript. This layer handle file uploads/downloads, error feedback, and password inputs.
3. **Controller:** This layer consists of backend route handlers and logic modules. It receives API requests from View, invokes logic of operations in the Model, and returns appropriate responses to View.

Use in Secdrop:

- **Model:** Classes or modules managing metadata and crypto operations (e.g. MetadataStore, EncryptionService in filesencryptdecrypt.js).
- **View:** React pages/components (client/src/Homepage.js, FileEncryptUpload.js, DownloadPage.js).
- **Controller:** Express routers in server/routes/files.js map HTTP verbs/paths to model operations, then return JSON or file streams .

MVC Pattern



Model includes logic and functionality of the application, and it is demonstrated in files:

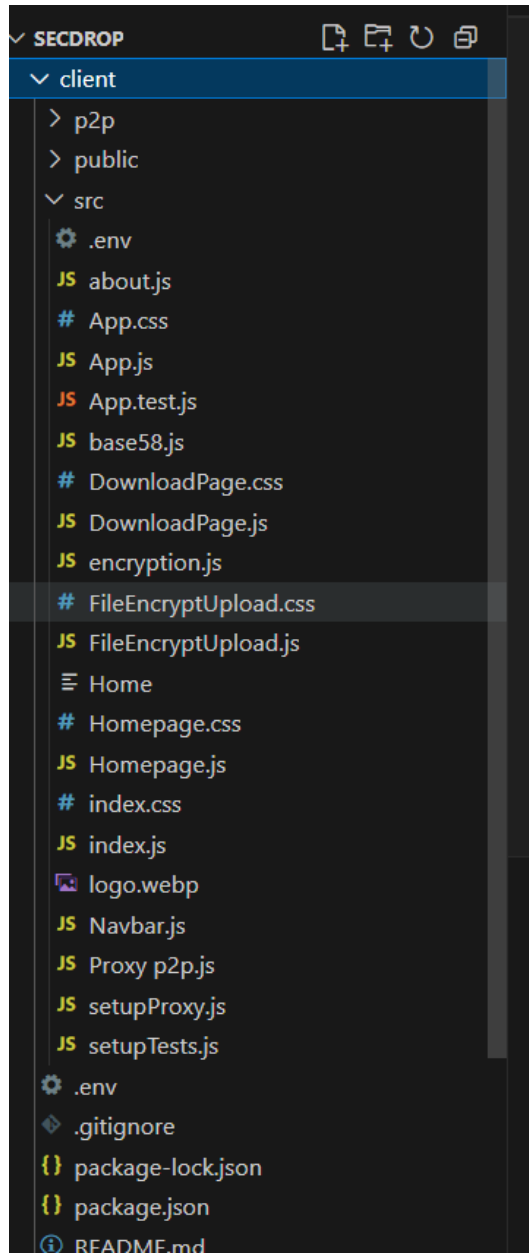
1. `/server/encryptdecrypt/filesencryptdecrypt.js`
2. `/server/aws/ec2.js`

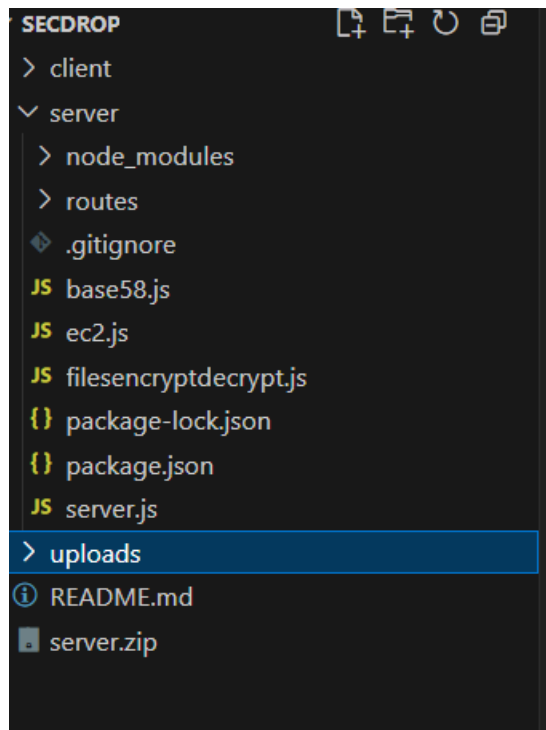
Controller handles API routes and calling model functions and it is demonstrated in file:

1. `/server/routes/files.js`

View handles file upload and download UI. It also interacts with backend APIs and prompt error feedback. It is demonstrated in directory:

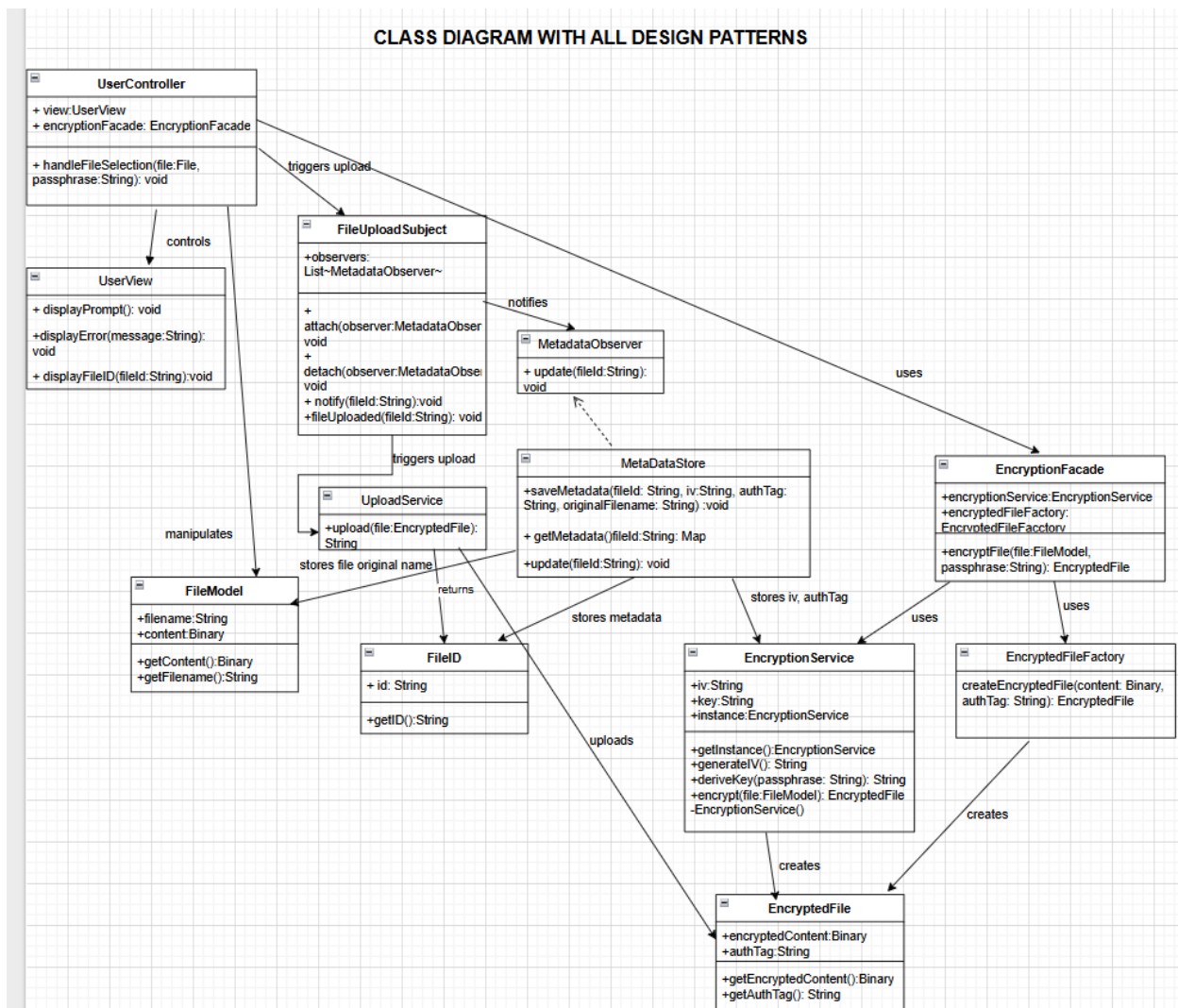
1. `/client/src/components/FileUpload.jsx`
2. `client/src/pages/HomePage.jsx`





The implementation of the MVC pattern in Secdrop has enhanced the modular architecture, clear separation of responsibilities, and simplified testability. As the UI, logic and data are clearly separated, it is easier to debug and make updates without affecting other layers which also increases scope of scalability. The breakdown into smaller components also enhances reusability as it can be reused across different parts of the application.

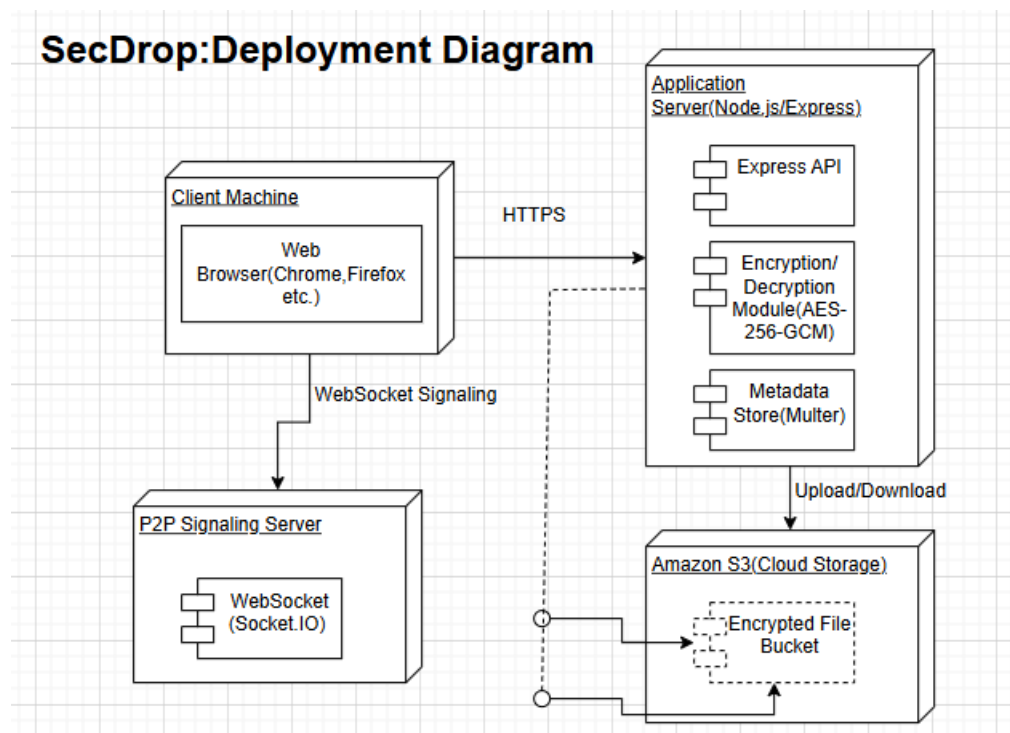
4.3 Class Diagram of Whole System Using Design Pattern



- The class diagram of the Secdrop incorporates multiple design patterns: Singleton, Facade, Factory, Observer, and MVC pattern to promote reusability, maintainability, modularity, and scalability across software components.
- The MVC pattern separates the system into three layers: the Model (MetadataStore, User, EncryptionService), the View (UserView) and the Controller (UserController). This separation provides clean code structure and modularity.
- The Singleton pattern is applied to class diagrams to control the upload size limits, encryption keys, and admin options. The design pattern prevents potential bugs and promotes consistency.
- The Factory Pattern abstracts the creation of secure tokens or password-protected links without exposing internal logic to controllers.
- The Observer Pattern is implemented in class MetadataObserver to monitor the activities without coupling code with file operations and improving maintainability and scalability.

- The Facade Pattern is implemented in EncryptionFacade which combines complex operations of encryption into simple API for controllers. This reduces redundancy and dependencies.
- To summarize, the class diagram shows how Secdrop implemented the patterns to structure a clean, robust, and secure file-sharing platform.

5. Deployment diagram



5.1 GitHub Link

[secdrop's link](#)

6. Programming Languages

- **JavaScript (Frontend & Backend)**-Used for building the client-side React components and backend APIs in Node.js.
- **HTML5 (Frontend)**-Provides the structure of web pages.
- **CSS3 (Frontend)**-Used for styling the user interface and ensuring responsiveness.

6.2 List of Reused Algorithms and Small Programs

- **AES-GCM encryption**-We rely on the standard AES-GCM cipher (authenticated encryption) provided by Node's crypto module on the server and the Web Crypto API in the browser. This gives us confidentiality and built-in tamper-detection.
- **PBKDF2 key derivation**-Passwords and tokens are stretched into 256-bit AES keys via PBKDF2 (HMAC-SHA256) with a fixed salt, using the same crypto libraries.
- **UUID Generation** generate unique IDs for naming and tracking files.
Location: /server/encryptdecrypt/filesencryptdecrypt.js.
- **Multer Middleware** -The Multer npm package handles multipart form-data parsing so we can receive file streams in Express with minimal boilerplate. Location: /server/routes/fileRoutes.js
- **dotenv loader**
The dotenv package reads our .env file at startup, keeping secrets and configuration out of code.

6.3 List of software tools and environments.

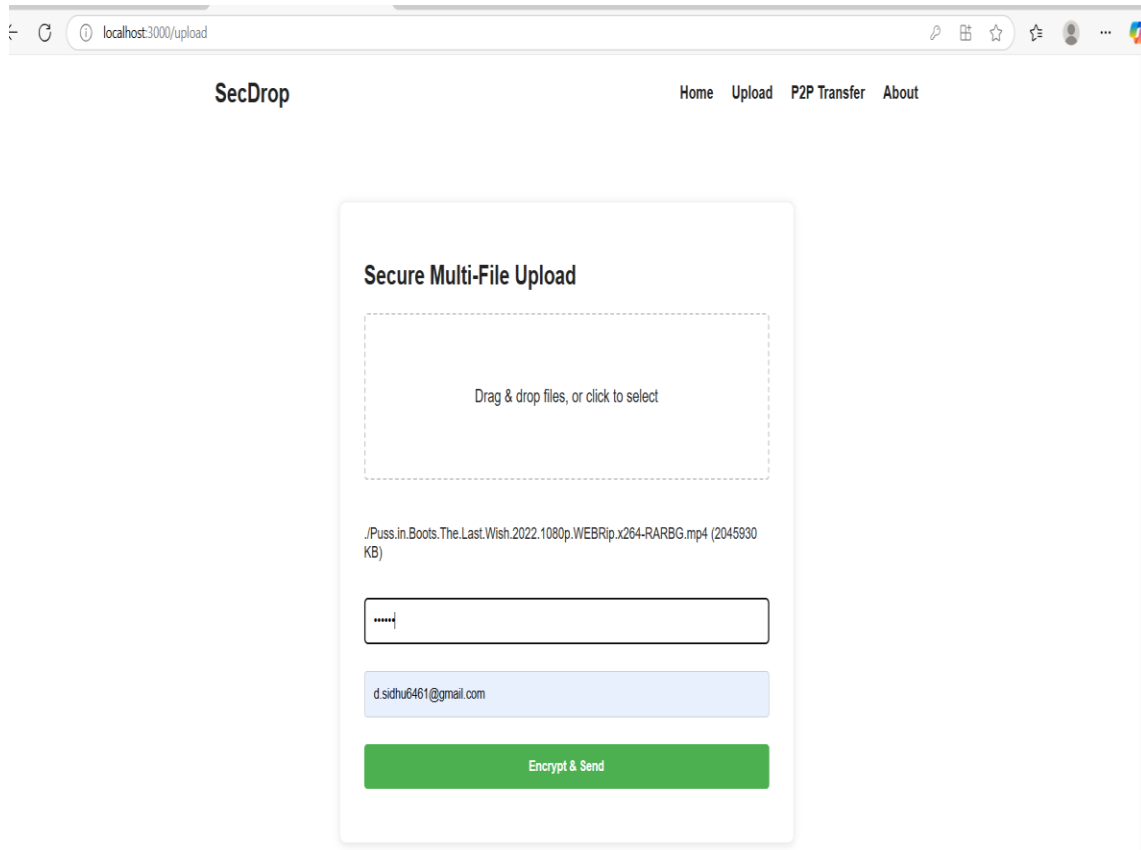
- **React.js** simplifies development of reusable components and ensures faster updates.
- **Node.js** with **Express** helps in efficient file processing.
- **Npm**-both client and server packages.
- **Express.js**-Web framework handling REST endpoints, middleware, and error routing.
- **Git**-Version control with feature branches and commit history.
- **VS Code** provides integrated terminal and git support.
- **AWS SDK for JavaScript v3**
Optional S3 client in ec2Storage.js for cloud file storage.
- **Web Crypto API**
Browser-native crypto for client-side key derivation and streaming encryption.

7. Software Testing

7.1 Correctness Testing

Test Case	Steps	Expected Result
CT-1: Encrypt & Upload	1. Open "Encrypt & Upload" page 2. Select a sample file (e.g. document.txt) 3. Enter a valid password (≥ 8 characters)	The file is sent to /api/crypto/upload, encrypted with AES-GCM, and the server returns a fileId.

4. Click **Encrypt & Upload**



The screenshot shows a web browser window with the address bar displaying 'localhost:3000/upload'. The page title is 'SecDrop'. The navigation menu includes 'Home', 'Upload', 'P2P Transfer', and 'About'. The main content area is titled 'Secure Multi-File Upload' and features a dashed box for file selection with the text 'Drag & drop files, or click to select'. Below this, a file is listed: './Puss.in.Boots.The.Last.Wish.2022.1080p.WEBRip.x264-RARBG.mp4 (2045830 KB)'. There is a password input field with a masked password '*****', an email input field containing 'd.sidhu6461@gmail.com', and a green 'Encrypt & Send' button.

SecDrop

Home Upload P2P Transfer About

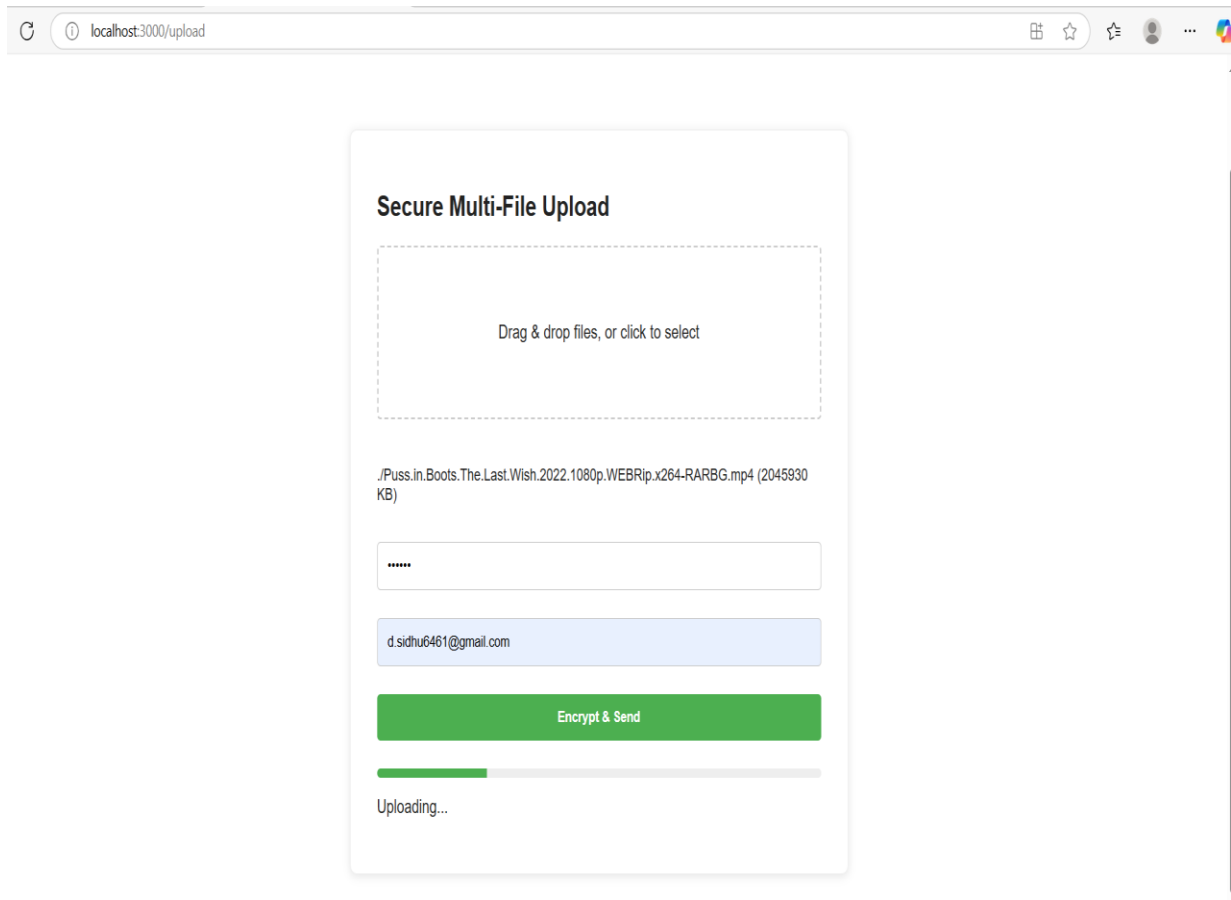
Secure Multi-File Upload

Drag & drop files, or click to select

./Puss.in.Boots.The.Last.Wish.2022.1080p.WEBRip.x264-RARBG.mp4 (2045830 KB)

d.sidhu6461@gmail.com

Encrypt & Send



Test Case	Steps	Expected Result
CT-2: Link & QR Generation	After CT-1 succeeds: 1. Verify the UI displays a download URL 2. Verify a QR code for that URL appears	A clickable download link and scannable QR code are shown.

Secure Multi-File Upload

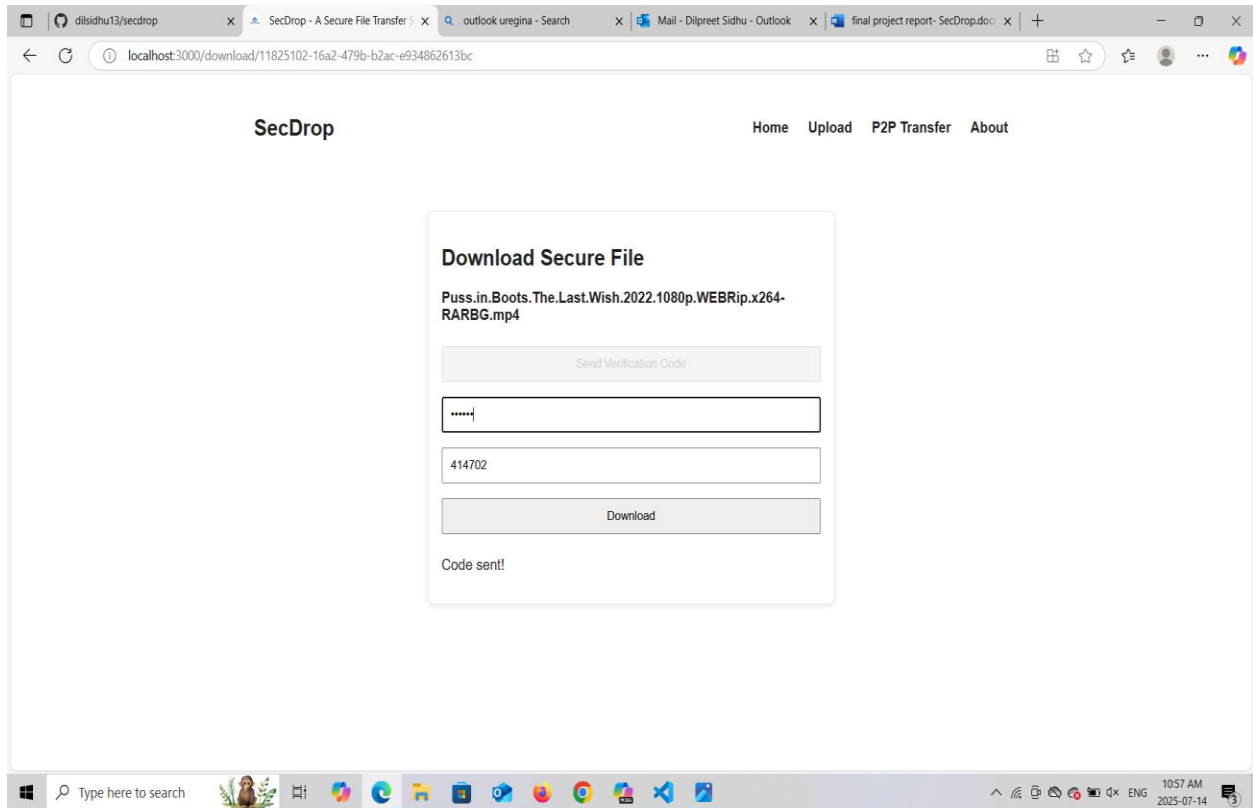
Upload complete!

Download Links

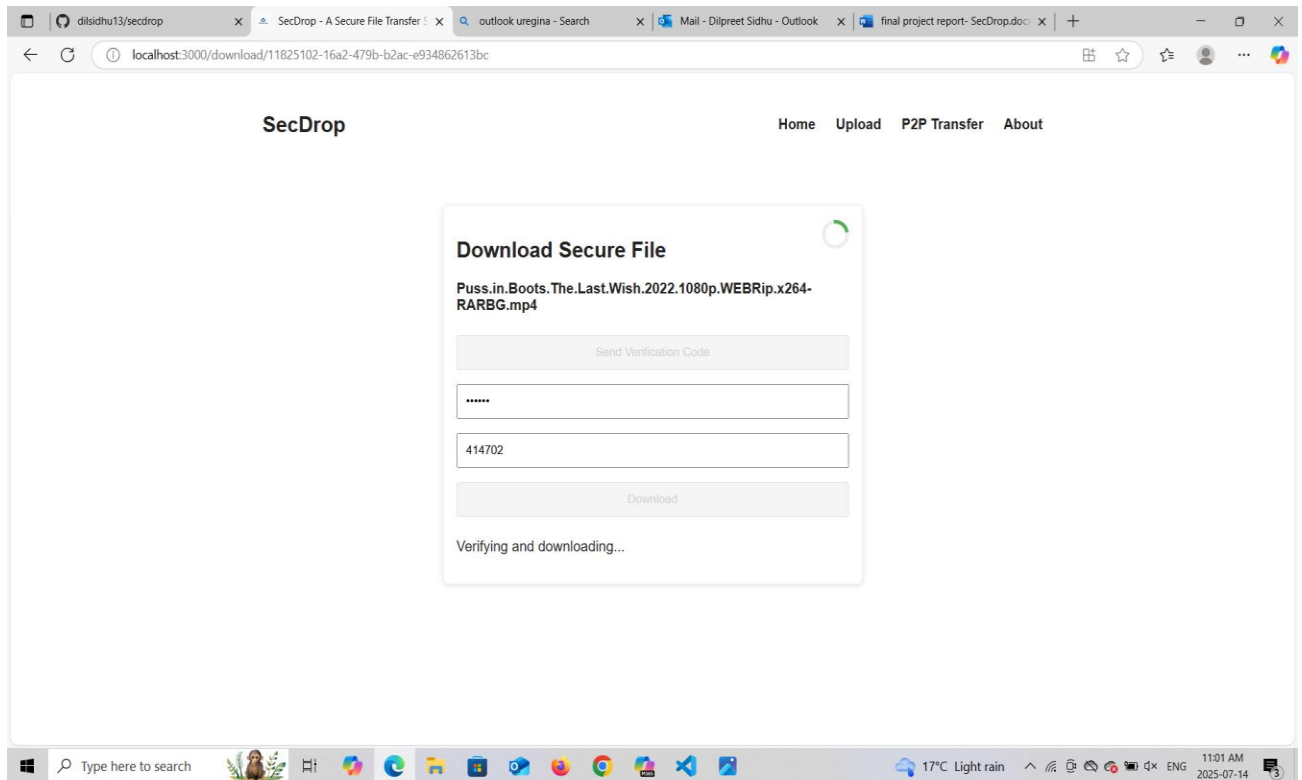
Puss.in.Boots.The.Last.Wish.2022.1080p.WEBRip.x264-RARBG.mp4:
<http://localhost:3000/download/11825102-16a2-479b-b2ac-e934862613bc>

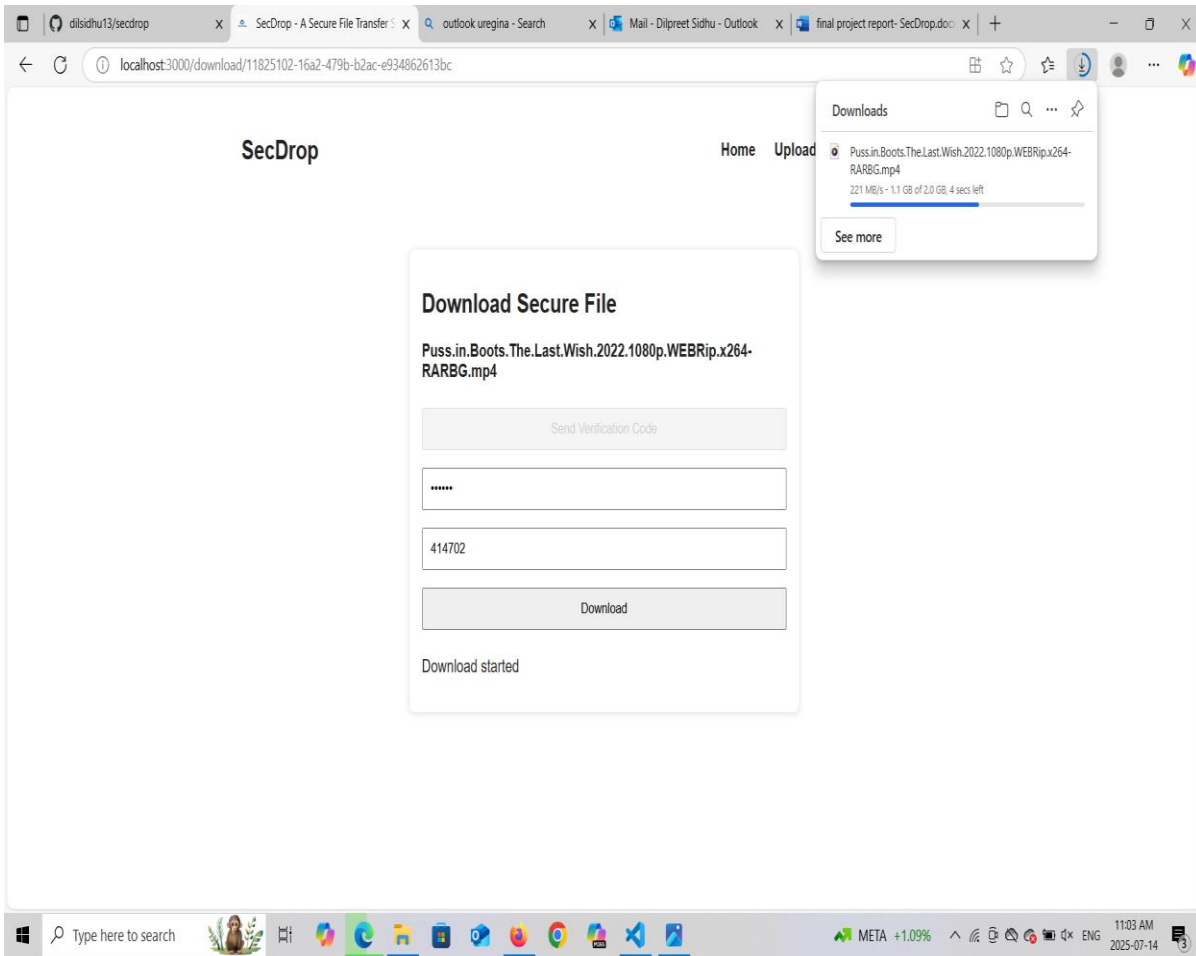


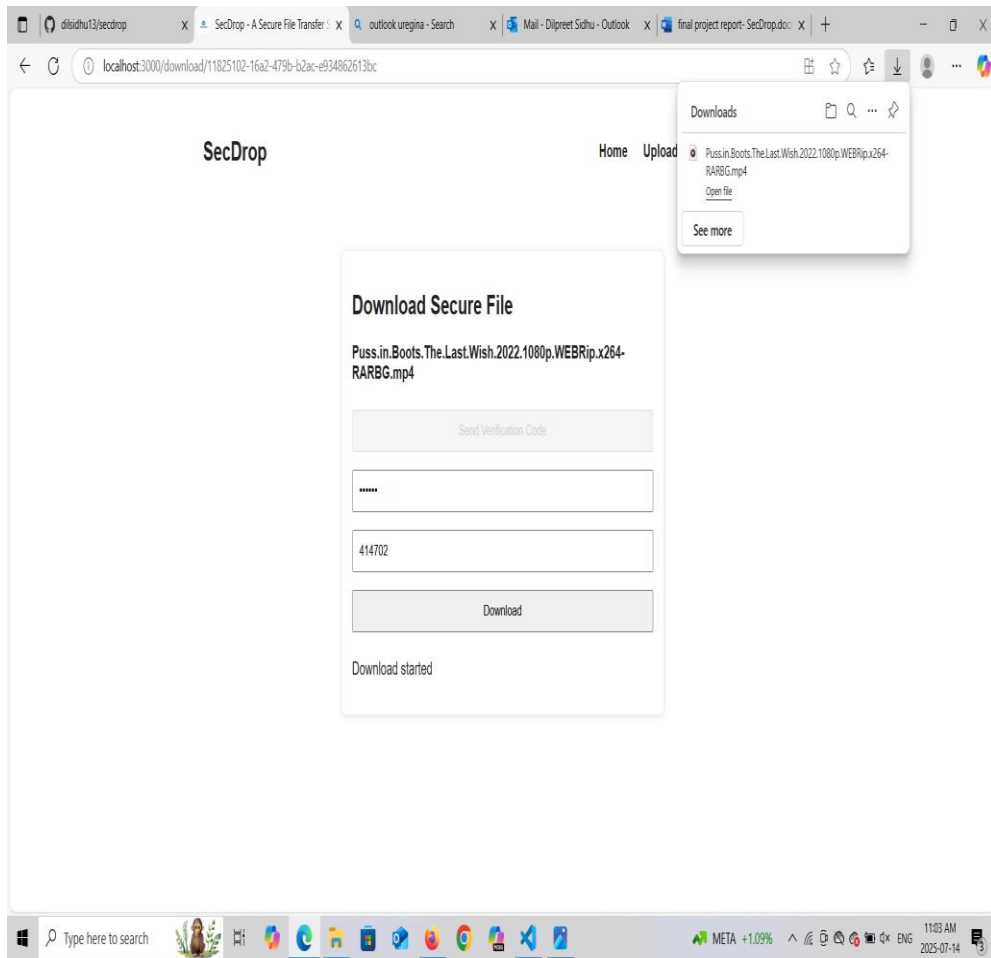
Test Case	Steps	Expected Result
CT-3: QR Code Scan	1. Scan the QR code with a mobile device or a desktop scanner 2. Follow the encoded URL	The secure download portal opens at /download/<fileId>.



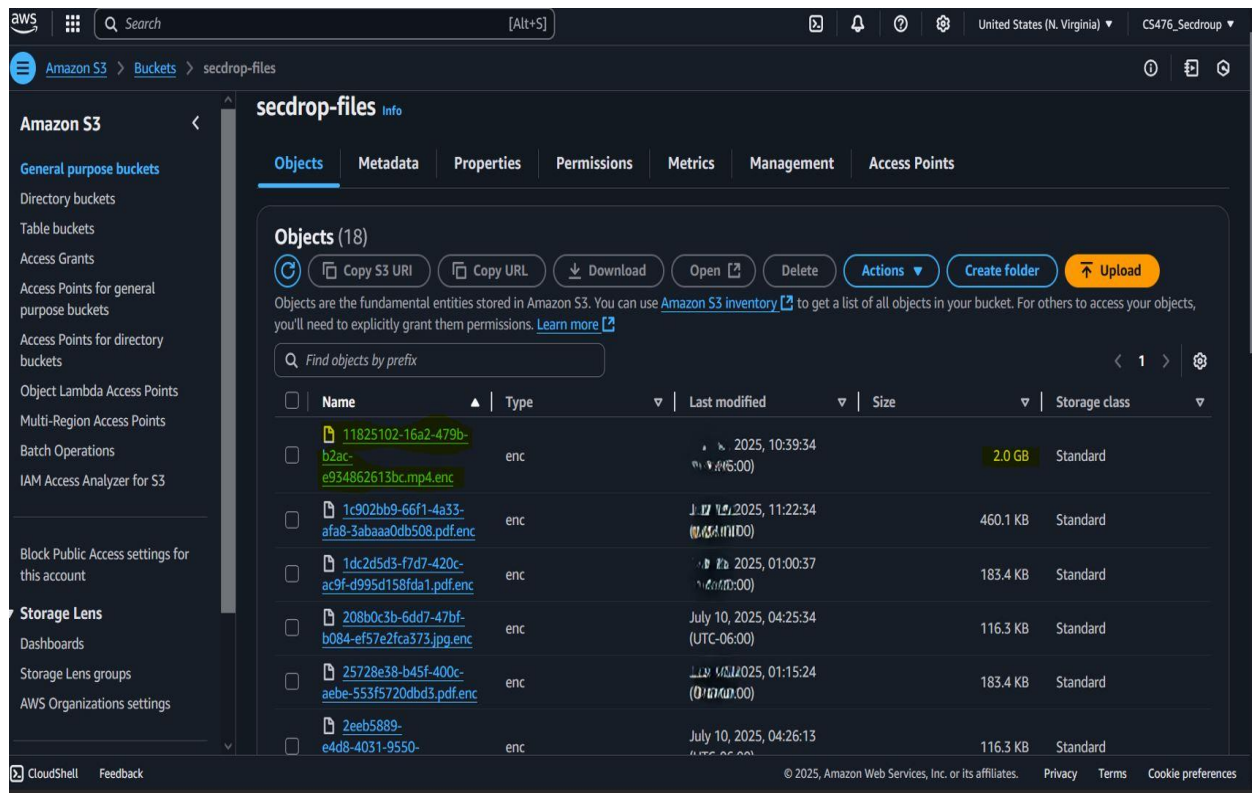
Test Case	Steps	Expected Result
CT-4: Decryption & Download	1. On the download page, enter the same password from CT-1 2. Click Download	The client sends POST /api/crypto/decrypt/:fileId, the server decrypts the blob, and the original file is offered for download







CLOUD SYSTEM SCREENSHOT: This screenshot shows the encrypted file on the Amazon Cloud-based system.



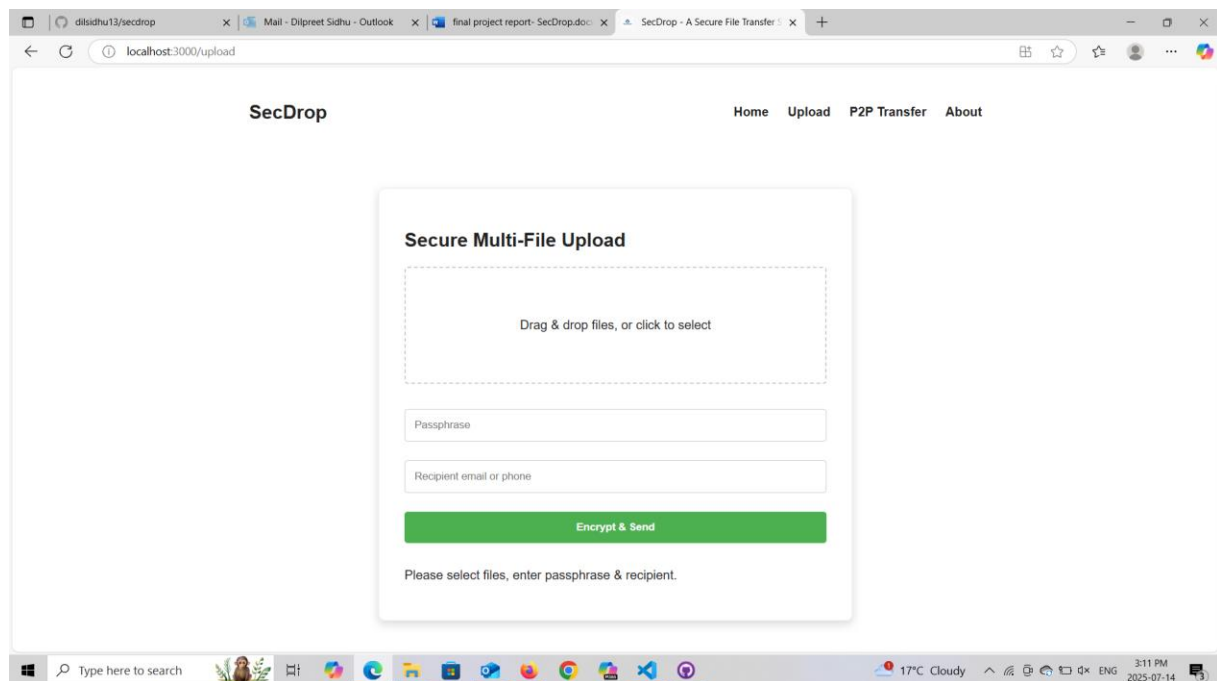
7.2 Robustness Testing

Uploading Robustness Testing (E2E) :

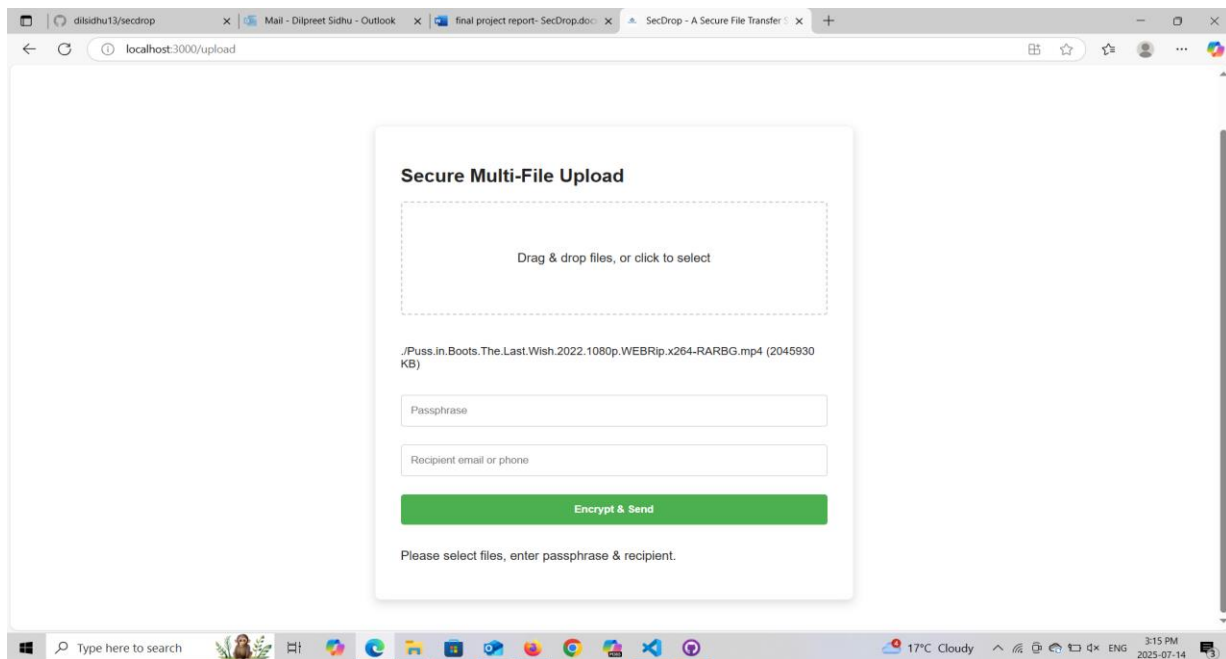
Test Case	Inputs	Behavior
UT-1: No inputs	- No file - No password (if present) No recipient email	Upload button stays disabled; user error shown.
UT-2: File only	- File selected - No password - No email	Validation error: "Please enter a password."
UT-3: Password only	- No file - Password entered - No email	Validation error: "Please select a file."
UT-4: Email only	- No file - No password - Email entered	Validation errors on file & password.

UT-5: Invalid email/password	File + password (< 8 chars) + malformed email (e.g. foo@)	Both “Invalid email” and “Password too short” messages display.
------------------------------	--	---

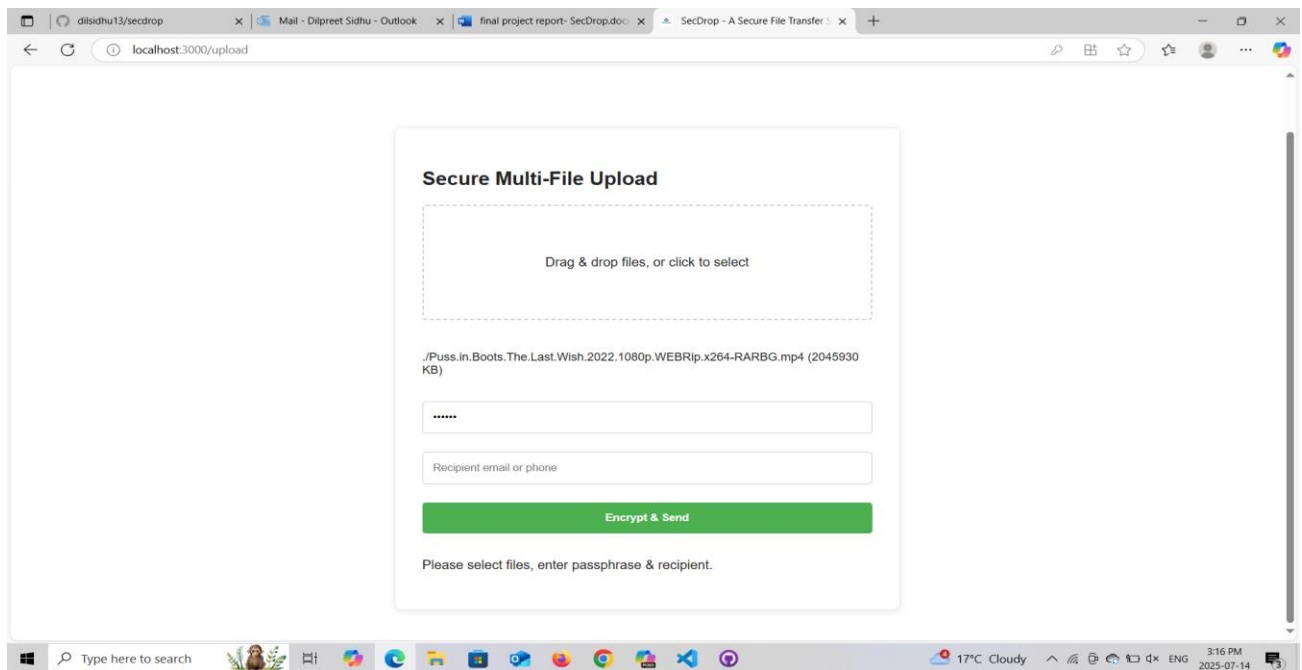
TEST CASE 1: No Files selected, No Passphrase input, No recipient email Input. And it will not proceed.



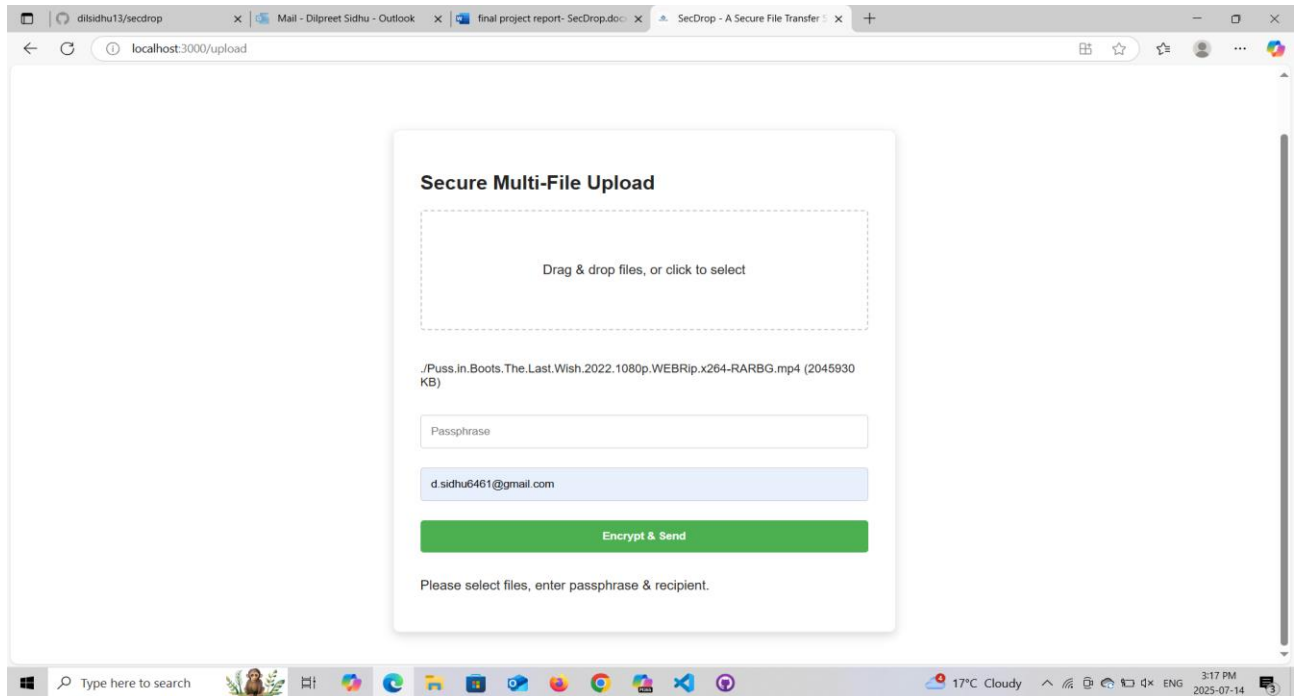
TEST CASE 2: File Selected but No Passphrase and no recipient email Input. And it will not proceed.



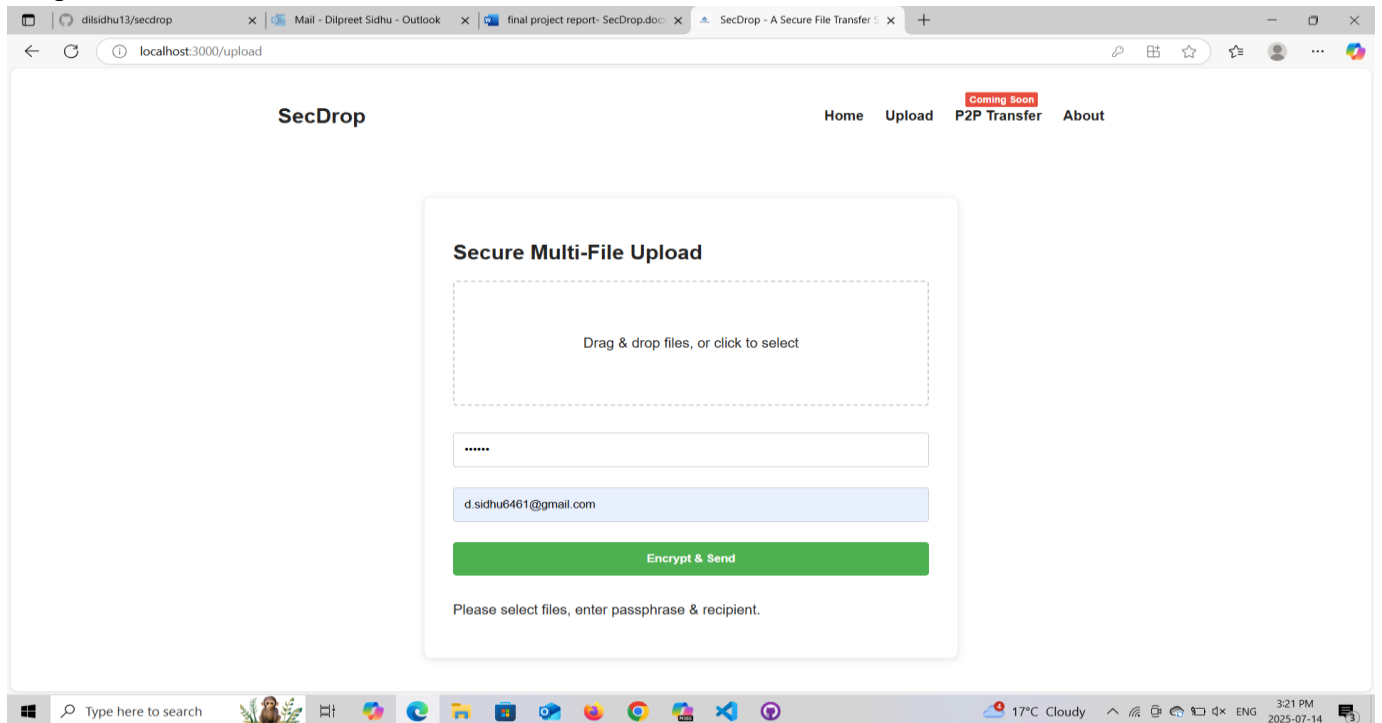
TEST CASE 3: File Selected and entered Passphrase but no recipient email Input. And it will not proceed.



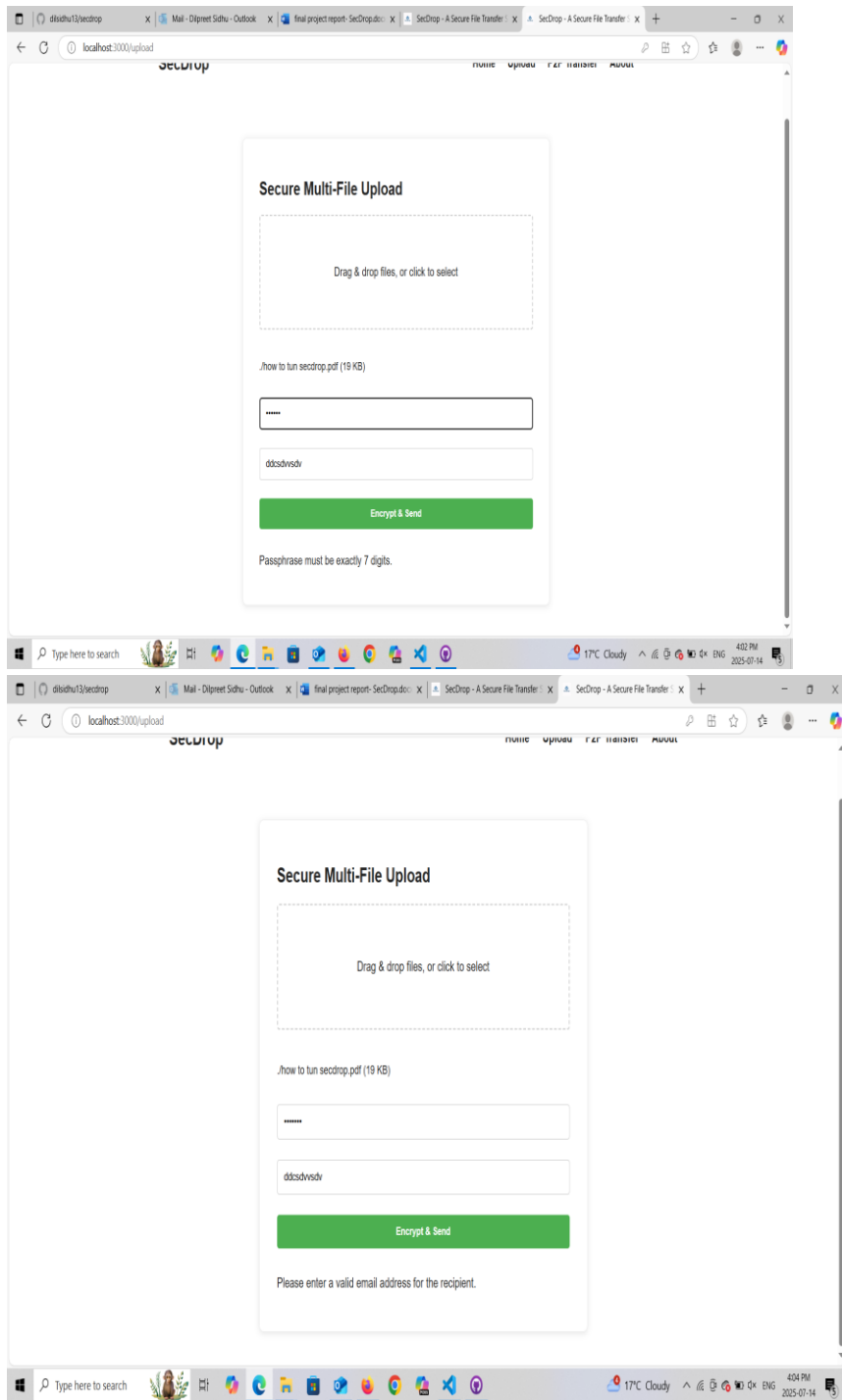
TEST CASE 4: File Selected but No Passphrase and recipient email Input. And it will not proceed.



TEST CASE 5: No File selected but entered Passphrase and recipient Email. As a Result, it will not proceed until.



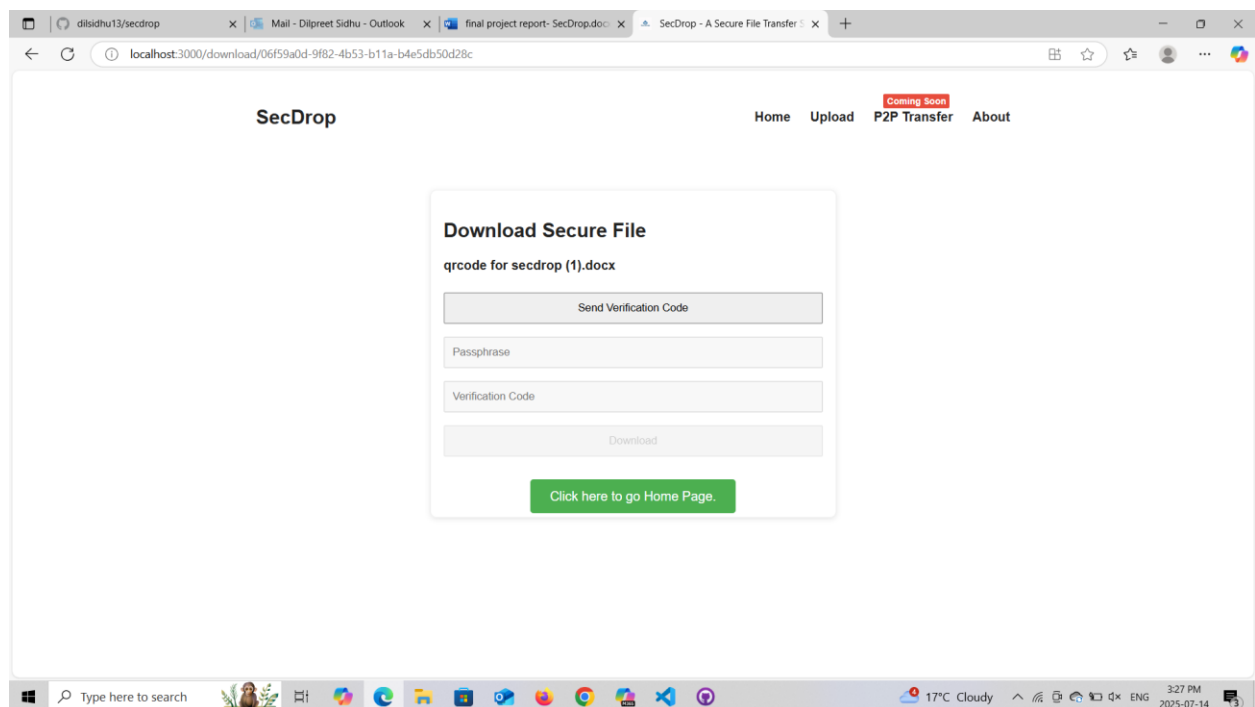
TEST CASE 6: Wrong email address and less than 7 decimal places passphrase.



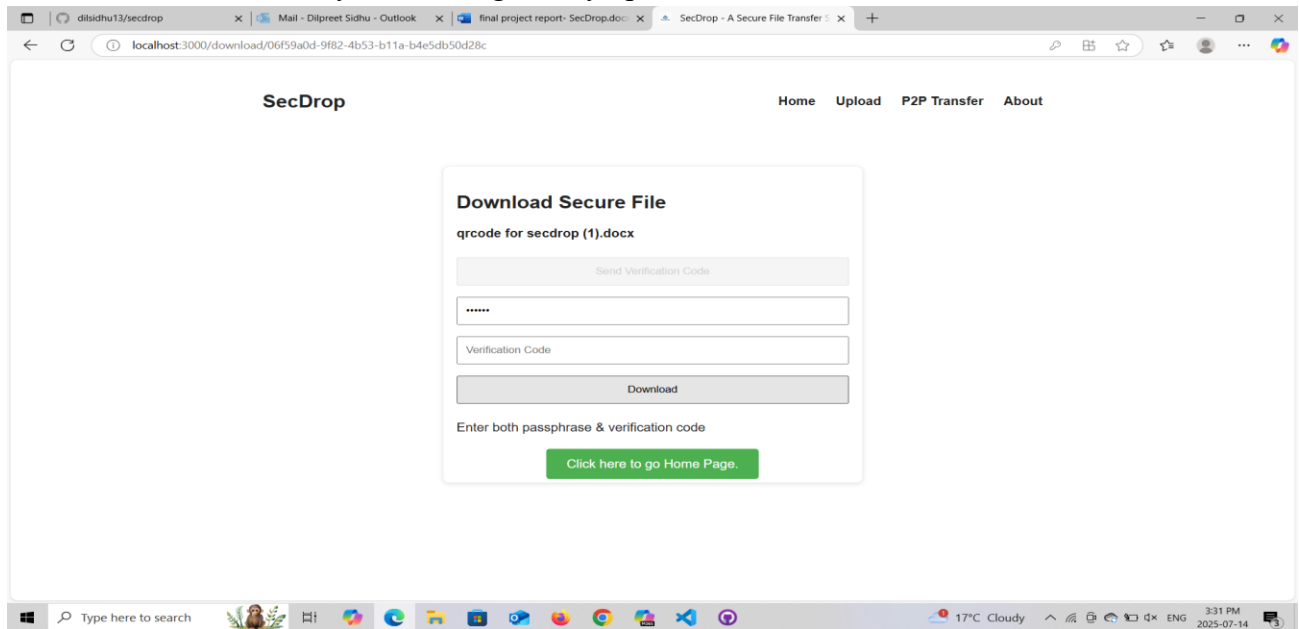
Downloading Robustness Testing:

Test Case	Steps	Behavior
DT-1: Missing verification	1. Navigate to download page without URL code/email verification	“Please verify your email” prompt appears; download blocked.
DT-2: Password only	1. Submit password without email code	Validation error: “Enter verification code.”
DT-3: Code only	1. Submit email code without password	Validation error: “Enter password.”
DT-4: Wrong credentials	1. Enter wrong password and/or wrong code	Server responds 401 Unauthorized; UI shows “Authentication failed.”

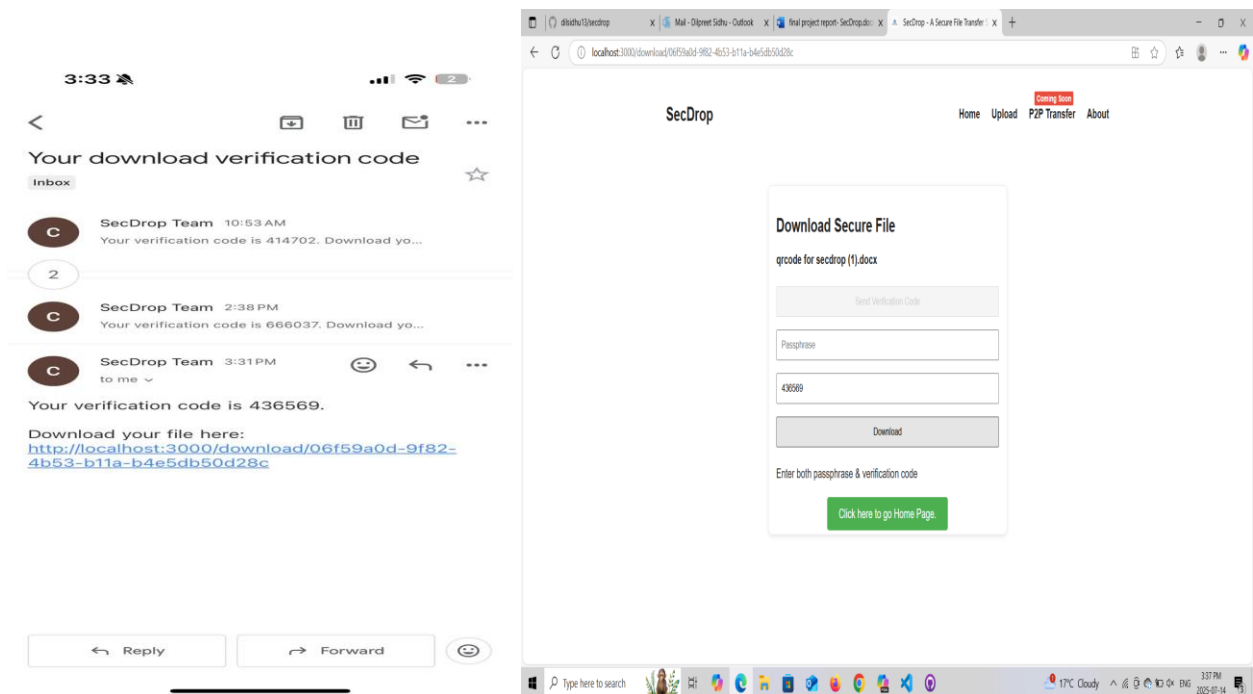
TEST CASE 1: For the Download, it will not allow you to enter or download anything without sending Verification code to the recipient email address.



Test Case 2 : After clicking, Send Code will not allowed you to download without email verification code with only Password given by uploader.

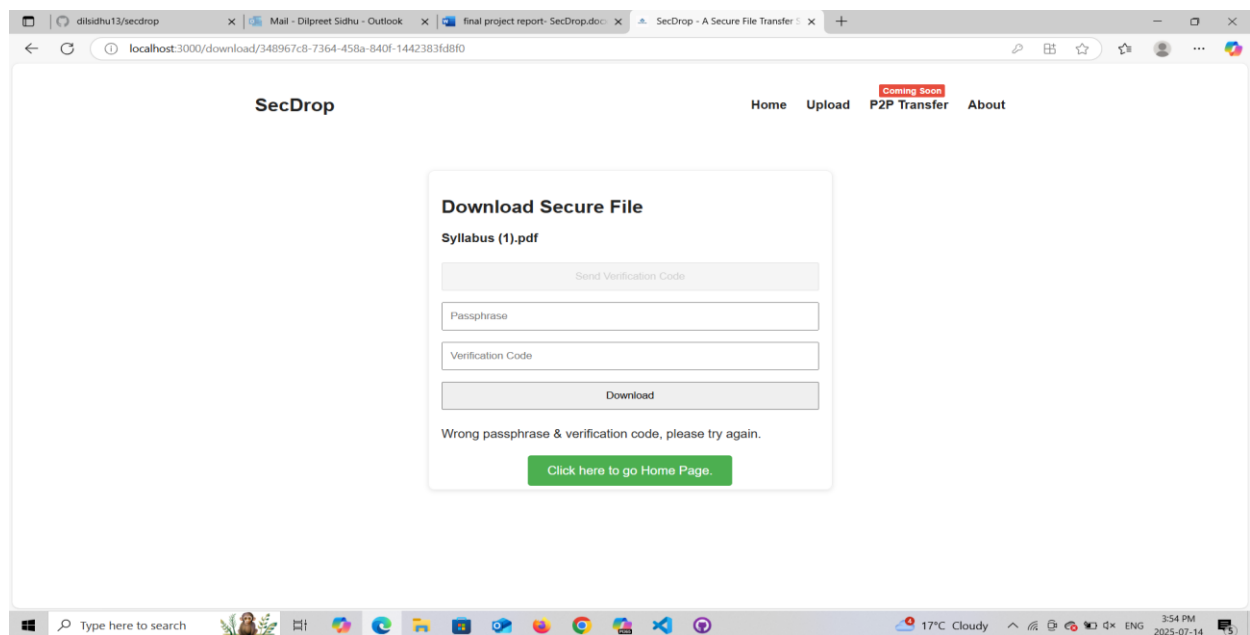


TEST CASE 3 : input the correct Verification code but no code



☐ ☆ > me	Your download verification code - Your verification code is 354
☐ ☆ > me	Your secure file is ready to download - Your file is ready! Ac
☐ ☆ > me	Your download verification code - Your verification code is 370
☐ ☆ > me	Your secure file is ready to download - Your file is ready! Ac
☐ ☆ > me	Your download verification code - Your verification code is 973
☐ ☆ > me	Your download verification code - Your verification code is 977
☐ ☆ > me	Your secure file is ready to download - Your file is ready! Acces
☐ ☆ > me	Your download verification code - Your verification code is 383
☐ ☆ > me	Your secure file is ready to download - Your file is ready! Ac
☐ ☆ > me	Your download verification code - Your verification code is 333
☐ ☆ > me	Your secure file is ready to download - Your file is ready! Ac

TEST Case 4: Wrong Passphrase & Verification Code.



7.3 Time-Efficiency Testing

Test Case	File Size	Action	Measured Time
TT-1	116 kb	Encrypt & upload	<pre> jenil_chaklasiya@JENILs-MacBook-Air server % npm start > server@1.0.0 start > node server.js SecDrop backend listening on port 4000 jenil_chaklasiya@JENILs-MacBook-Air server % npm start > server@1.0.0 start > node server.js SecDrop backend listening on port 4000 Encrypt & Upload: 2.323ms Download & Decrypt: 486.966ms </pre>
TT-2	8.9 MB	Encrypt & upload	<pre> jenil_chaklasiya@JENILs-MacBook-Air server % npm start > server@1.0.0 start > node server.js SecDrop backend listening on port 4000 Encrypt & Upload: 15.02ms Download & Decrypt: 8.596s </pre>
TT-3	300 mb	Download & decrypt	<pre> Download & Decrypt: 8.596s Encrypt & Upload: 325.823ms Download & Decrypt: 31.945s </pre>

8. Conclusion

Secdrop provides a lightweight, end-to-end secure file sharing platform that leverages AES-GCM encryption with PBKDF2-derived keys to protect every upload and download. Its clean three-layer architecture—React-based frontend for user interaction, Express.js logic layer for crypto and routing, and a pluggable storage layer supporting both local disk and AWS S3—ensures clear separation of concerns and easy maintainability.

With features like multi-file upload, downloadable links, and QR-code generation, users can quickly encrypt and share files without exposing raw data. Comprehensive Jest unit tests and Supertest end-to-end scripts validate all core workflows, while time-efficiency measurements

confirm that typical files (up to 5 MB) encrypt, upload, decrypt, and download in about few second on commodity hardware.

Looking forward, Secdrop can be extended with chunked or resumable uploads for very large files, user authentication for access control and audit trails, and optional client-side key escrow for recovery. These enhancements will further strengthen security and usability, building on the solid foundation of our current implementation.